

LEWAM: JEPA CAN BE SUCCINCT FOR REALISTIC CONTROL

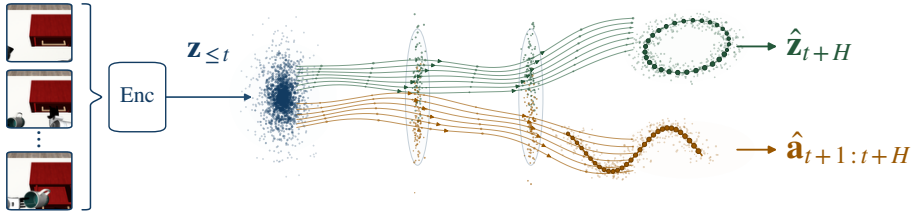
Anonymous authors

Paper under double-blind review

ABSTRACT

Joint-embedding predictive architectures (JEPAs) offer compact, reconstruction-free world models for control. Existing JEPA controllers, however, often combine specialized components: auxiliary anti-collapse objectives, separately trained action policies or population-based solvers, and pretrained visual backbones. These choices make the resulting pipelines complex to train and expensive to deploy. Despite this added machinery, their demonstrated scope remains largely limited to low-complexity, short-horizon tasks. A terminal goal specifies what the world should become, but not how the agent should act to make it so. To address these issues, we introduce **LeWAM**, a succinct JEPA with no auxiliary components or constraints: a single joint prediction objective is sufficient to learn control-centric representations end-to-end. Under this unified objective, forward dynamics describes what happens, while action prediction specifies how to realize it. To scale up LeWAM test-time planning, we further introduce Steer-MPC, which performs search by optimizing a bounded goal-latent steering vector while keeping the predictor frozen. Across 8 simulated environments and 5 physical-robot tasks, LeWAM achieves high success rates in contact-rich manipulation and long-horizon control—a task scope beyond prior JEPA benchmarks. Representation probes further show that control-centric physical structure emerges even without anti-collapse regularization.

(a) LeWAM Training



Joint Flow Matching

(b) LeWAM Planning

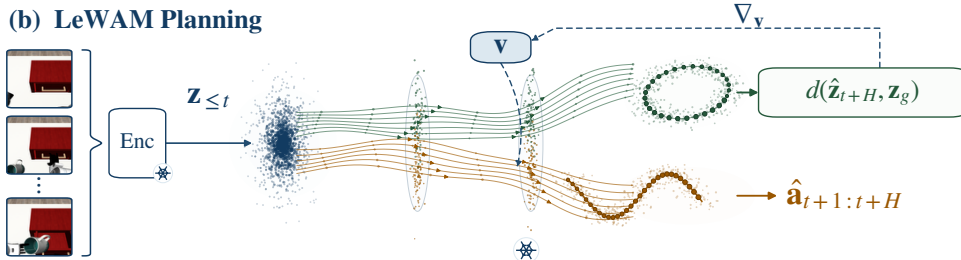


Figure 1: **One joint predictor for learning and planning.** (a) LeWAM maps history latents to joint future latent and action trajectories. (b) Steer-MPC freezes the predictor and updates only a bounded steering vector; the terminal latent cost closes the loop and the first predicted action is executed.

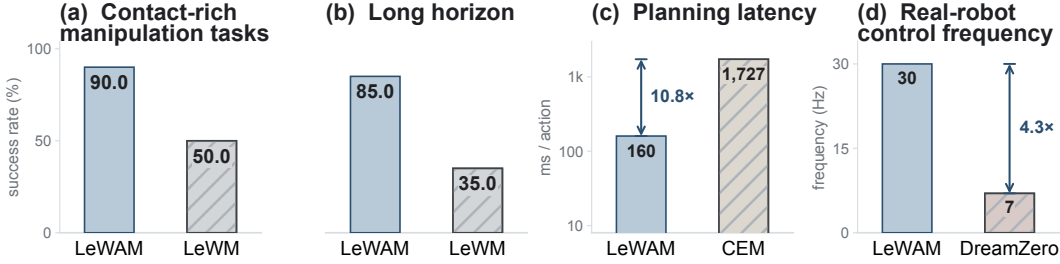


Figure 2: **Task-complete control and efficiency.** Average success on contact-rich manipulation and task-completion benchmarks (left), planning latency against CEM, and real-robot control frequency (right). DreamZero’s published optimized rate is included as a cross-system rather than matched-hardware reference.

1 INTRODUCTION

World models learn how actions change the world. Rather than reconstruct future pixels, JEPAs model action-conditioned physical dynamics in latent space (Hafner et al., 2019; Bardes et al., 2024; Maes et al., 2026b). Control additionally requires executable action generation and planning.

Current JEPA systems obtain representation stability, action generation, visual knowledge, and planning through specialized objectives, heads, pretrained backbones, or online search. World prediction, policy, and planner thus remain fragmented, while evaluation centers on low-complexity, short-horizon goal reaching. Can one end-to-end JEPA provide latent simulation, direct control, and efficient planning for contact-rich and long-horizon tasks?

A terminal goal specifies what the world should become, but not how the agent should act to make it so. Forward-model planners recover this missing action continuation online through CEM (Rubinstein, 1999), MPPI (Williams et al., 2017), iCEM (Pinneri et al., 2021), or Gradient MPC (Jyothir et al., 2023). This is particularly demanding in contact-rich manipulation (Agrawal et al., 2016; Posa et al., 2014; Pang et al., 2023), where success depends on precise, temporally coordinated contact strategies. We instead learn the action continuation together with its resulting latent evolution.

We introduce **LeWAM**, which predicts future latent states and actions through one end-to-end conditional-flow objective. Its action flow models executable behavior, while its latent flow is conditioned on clean actions and learns the resulting physical change. Their joint target couples how behavior unfolds with what it causes, leveraging the temporal structure of demonstrations without reducing control to trajectory replay. Rather than preserving every observable distinction, this objective pressures the encoder to retain distinctions that change demonstrated actions or their predicted physical consequences. This task-induced supervision removes the need for an auxiliary SIGReg prior, a post-trained action head, pixel reconstruction, or a pretrained visual backbone. **LeWAM is succinct at the system level, but expressive at the objective level.**

The frozen predictor can simulate action-conditioned futures in latent space, act directly through its action marginal, and support stable and efficient test-time planning without parameter updates. Steer-MPC optimizes a bounded steering variable through this predictor, searching its learned continuation family instead of assigning each time step an independent raw-action coordinate.

We first evaluate short-horizon goal reaching to intermediate trajectory states rather than task endpoints, testing whether LeWAM follows the requested outcome instead of defaulting to demonstration completion. We then evaluate contact-rich manipulation, long-horizon task completion, and physical-robot control. Matched objective ablations and planner comparisons test the joint predictor and its success–compute frontier.

2 RELATED WORK

World Models. World models learn predictive environment dynamics from data, enabling agents to reason about future states before execution (Ha & Schmidhuber, 2018). Latent formulations

SIGReg, Stop-grad, EMA	GC-IDM, PRISM	DreamZero, Fast-WAM	CEM, MPPI, iCEM	LeWM, DINO-WM, PLDM
⊗ Auxiliary anti-collapse	⊗ Action-head post-training	⊗ Generative Video WAMs	⊗ Population-based search	⊗ Short-horizon simple tasks
LeWAM				
✓ No auxiliary anti-collapse	✓ No action post-training	✓ No reconstruction ✓ No pre-training	✓ Scalable gradient steering	✓ Contact-rich tasks ✓ Long horizon

Figure 3: **System-level positioning.** Column-wise comparison with auxiliary anti-collapse design, action-head post-training, generative video WAMs, population-based search, and short-horizon simple tasks.

make this reasoning tractable: PlaNet (Hafner et al., 2019) plans in a compact latent state space, Dreamer (Hafner et al., 2020) learns behavior from imagined rollouts, and DreamerV3 (Hafner et al., 2023) scales this paradigm across domains. Token-based models such as IRIS (Micheli et al., 2023) extend predictive modeling to longer visual sequences, while Genie (Bruce et al., 2024) learns interactive video environments. For control, TD-MPC2 (Hansen et al., 2024) combines latent dynamics, reward and value learning, and policy-guided trajectory optimization. Many control-oriented world models therefore use reward or value supervision to shape imagined behavior. We instead study reward-free predictive control, where representations and dynamics are learned without either signal.

JEPA. JEPA predicts the evolution of a system in a compact latent space without reconstructing observations. I-JEPA (Assran et al., 2023) instantiated this principle for images, and V-JEPA (Bardet et al., 2024) extended it to video. LeVJEPA (Kuhn et al., 2026) pairs view invariance with SIGReg (Balestriero & LeCun, 2025) to train a single video encoder for broadly transferable appearance and motion features. DINO-WM (Zhou et al., 2024) then modeled action-conditioned dynamics over frozen DINOv2 features for visual planning. In 2025, PLDM (Sobal et al., 2025) jointly learned visual representations and latent dynamics with VICReg-inspired and inverse-dynamics regularization, while V-JEPA 2-AC (Assran et al., 2025) post-trained an action-conditioned predictor on a video-pretrained encoder. LeWM (Maes et al., 2026b) subsequently demonstrated stable end-to-end JEPA training from pixels with a prediction-SIGReg objective and one regularization coefficient. **Action-augmented JEPAs** introduce direct action supervision into this line in complementary ways. GC-IDM (Nguyen et al., 2026) post-trains a goal-conditioned inverse-dynamics model on a frozen LeWM to amortize planning. SMWM (Ivashkov et al., 2026) trains a single-step inverse head to recover the executed action from consecutive embeddings and uses this auxiliary loss as its sole anti-collapse regularizer. Delta-JEPA (Zhang et al., 2026b) decodes actions from latent differences, whereas PRISM (Wang et al., 2026c) post-trains an action prior to guide sampling during latent planning. LeWAM asks a complementary representation question: it organizes latent structure through action and physical-consequence prediction for control, without an explicit latent-distribution regularizer or a separately post-trained inverse controller.

World Action Models. Early systems such as UVA (Li et al., 2025), UWM (Zhu et al., 2025), and WorldVLA (Cen et al., 2025) coupled predictions of future world states with executable actions. DreamZero (Ye et al., 2026) later generated future video and actions with a pretrained video diffusion model, and Fast-WAM (Yuan et al., 2026) retained generative video supervision while bypassing video synthesis during action inference. Representation-centric variants subsequently avoided pixel reconstruction: Being-H0.7 (Luo et al., 2026) aligns future-informed queries, while RepWAM (Wang et al., 2026a) learns semantic visual-action tokens. In contrast, LeWAM trains a compact joint latent-state/action continuation model end-to-end from pixels and retains both its action marginal for direct control and its joint rollout for imagination.

Planning with Latent Dynamics. CEM (Rubinstein, 1999), MPPI (Williams et al., 2017), iCEM (Pinneri et al., 2021), and Gradient MPC (Henaff et al., 2017; Jyothis et al., 2023) numerically optimize the action sequence induced by a learned forward model. CEM and MPPI use population-based derivative-free search, iCEM adds colored-noise correlation and elite reuse, and Gradient

MPC differentiates the same rollout objective. Contact makes this shared inverse multimodal and ill-conditioned. Sampling can cross modes but is rollout intensive; gradients avoid a population yet face nonsmooth rollouts and unsupported actions (Parthasarathy et al., 2025). Alternatives use value surrogates (Destrade et al., 2026), reshape latent costs (Wang et al., 2026b), shorten the horizon (Zhang et al., 2026a), or learn action priors (Wang et al., 2026c) and latent planners (Masip et al., 2026). LeWAM instead amortizes state–action continuations, and Steer-MPC searches within that frozen learned family.

3 METHODOLOGY

Problem and Notation. We learn from reward-free trajectories $\tau = (\mathbf{o}_{1:T}, \mathbf{a}_{1:T})$, where the action $\mathbf{a}_t$ induces the transition from $\mathbf{o}_t$ to $\mathbf{o}_{t+1}$. At time t , the observed history is $(\mathbf{o}_{\leq t}, \mathbf{a}_{\leq t})$. A context vector $\mathbf{c}_t$ may encode a goal or task instruction when one is available.

LeWAM. One encoder and one predictor define the state–action continuation:

$$\boxed{\mathbf{z}_t = \text{Enc}(\mathbf{o}_t), \quad \mathbf{z}_{t+1} = \text{Enc}(\mathbf{o}_{t+1}), \quad (\hat{\mathbf{z}}_{t+1}, \hat{\mathbf{a}}_{t+1}) \sim p_{\Theta}(\mathbf{z}_{t+1}, \mathbf{a}_{t+1} \mid \mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{c}_t)} \quad (1)$$

The model therefore assigns one distribution to the next latent state and action. Recursively applying the same predictor produces a joint continuation of imagined latent states and actions. *Architecture placeholder.*

The Terminal-Goal Inverse. Let $F_{\Theta}^{(H)}$ denote an H -step latent rollout under a proposed action continuation $\mathbf{A} = \mathbf{a}_{t+1:t+H}$. Latent-goal planning solves the inverse problem

$$\mathbf{A}^* \in \arg \min_{\mathbf{A}} d\left(F_{\Theta}^{(H)}(\mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{A}), \mathbf{z}_g\right). \quad (2)$$

CEM (Rubinstein, 1999), iCEM (Pinneri et al., 2021), and MPPI (Williams et al., 2017) approximate Equation 2 with sampling- and population-based derivative-free optimizers, whereas Gradient MPC (Henaff et al., 2017; Jyothir et al., 2023) differentiates the same rollout objective with respect to $\mathbf{A}$. The optimization method changes, but the geometry of the inverse problem does not.

Two properties of this geometry matter for contact-rich control. First, a terminal goal underdetermines how it is realized: distinct temporally coordinated action continuations may reach the same terminal latent or a cost-equivalent outcome. Second, contact makes the objective nonsmooth and mode-sensitive: nearby action proposals may cross a contact boundary at different times and enter different no-contact, sticking, sliding, or separation mode sequences, causing their latent continuations to diverge. Contact-mode growth, stiff dynamics, discontinuous or exploding gradients, and nonconvexity (Posa et al., 2014; Suh et al., 2022; Pang et al., 2023) compound this difficulty.

These properties do not make CEM incapable of contact-rich planning. As a black-box optimizer, it can search across modes, but it must reconstruct a valid inverse solution at test time by repeatedly rolling raw-action populations through the forward model. iCEM (Pinneri et al., 2021) reduces this cost with colored noise and elite reuse, but its temporal correlation is generic rather than conditioned on demonstrated behavior at the current history and goal. Gradient MPC (Parthasarathy et al., 2025) removes the population, yet directly faces nonsmooth rollout gradients and unsupported action iterates.

Joint Prediction Couples Goal, Action, and Consequence. An action model and a forward model address complementary sides of control. The former can amortize executable behavior from demonstrations, while the latter predicts how candidate actions change the world but otherwise leaves their terminal-goal inverse to online search. LeWAM learns both in one joint distribution, which admits the factorization

$$p_{\Theta}(\mathbf{Z}, \mathbf{A} \mid h_t, \mathbf{c}_t) = \pi_{\Theta}(\mathbf{A} \mid h_t, \mathbf{c}_t) F_{\Theta}(\mathbf{Z} \mid h_t, \mathbf{A}), \quad (3)$$

where $h_t = (\mathbf{z}_{\leq t}, \mathbf{a}_{\leq t})$, $\mathbf{A} = \mathbf{a}_{t+1:t+H}$, and $\mathbf{Z} = \mathbf{z}_{t+1:t+H}$. This factorization assigns each variable a distinct role: the requested goal conditions the action flow, whereas the goal-blind latent dynamics receives the clean action continuation during training and learns its resulting latent evolution. The goal therefore cannot alter an imagined future directly; it must first change the generated action. A controller that always defaults to episode completion cannot fit training windows whose requested goal is an intermediate state and whose demonstrated continuation is different. At

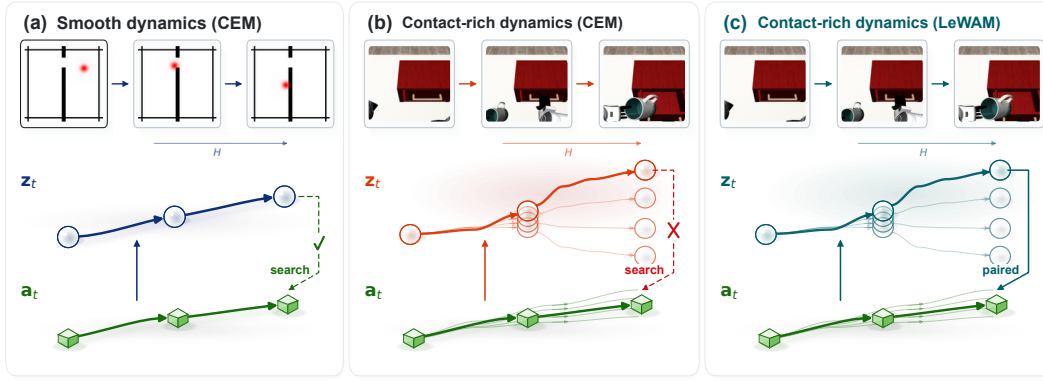


Figure 4: **Contact complicates the terminal-goal inverse.** (a) Under smooth dynamics, nearby actions induce consistently ordered latent outcomes, so CEM can recover a useful action through forward-rollout search. (b) In contact-rich dynamics, nearby proposals can cross contact boundaries at different times and enter different mode sequences; the resulting latent rollouts diverge over horizon H , making raw-action inversion discontinuous and multimodal. (c) Joint prediction leaves the same contact branches intact, but represents alternative solutions as paired latent-action continuations. The solid return path highlights one selected pair; it does not imply that the inverse has become unique. The cross denotes difficult raw-action inversion rather than impossible planning.

inference, the same latent dynamics evaluates generated or steered actions within the learned support. Every action continuation therefore remains paired with its predicted physical consequence. This echoes the complementary roles identified in Learning to Poke (Agrawal et al., 2016): inverse prediction encourages interaction-sensitive features, while forward prediction grounds that feature space in physical change.

The terminal-goal inverse remains set-valued: several coherent action continuations can reach the same goal (Agrawal et al., 2016). LeWAM represents these alternatives as modes $\{(Z^{(k)}, A^{(k)})\}_k$ rather than forcing a unique inverse. Conditional flow matching models the alternatives as a conditional distribution (Lipman et al., 2023), which is important because robot demonstrations can induce multimodal, temporally correlated action distributions that are poorly captured by scalar regression (Chi et al., 2023). Multimodality alone is therefore not the distinction: because the prediction target in Equation 3 is joint, each sampled action mode remains coupled to its own latent consequence. The action-conditional amortizes where coherent, demonstrated solutions lie (Amos, 2022), and the action-conditioned latent dynamics evaluates how each candidate changes the world. Thus, LeWAM does not remove inverse non-identifiability; it amortizes a behavior-informed multimodal proposal together with a predictive consequence model.

Control-Centric Structure. Prior joint-embedding methods prevent constant representations with explicit anti-collapse objectives such as VICReg (Bardes et al., 2022), SIGReg (Balestrierio & LeCun, 2025; Maes et al., 2026b), and VISReg (Wu et al., 2026). SMWM (Ivashkov et al., 2026) provides a particularly relevant precedent: its single-step inverse-dynamics head reconstructs the executed action a_t from consecutive embeddings (z_t, z_{t+1}) , and the separately weighted inverse loss backpropagates into the encoder as its sole anti-collapse mechanism. This supports the broader principle that action prediction can preserve action-relevant visual distinctions. LeWAM shares this principle but not the IDM construction: it predicts the future action continuation directly from history and the requested goal

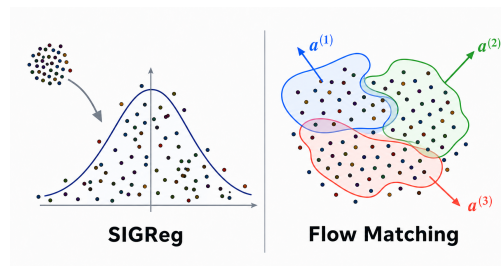


Figure 5: **Representation shaping.** Left: explicit latent-distribution regularization. Right: control-centric separation induced by joint action and latent targets.

within the same joint likelihood as its latent continuation, rather than recovering an intervening action from two consecutive latent states. The objective imposes no global latent marginal and does not require every visual distinction to survive. Instead, its two targets supply complementary control-centric pressures. If two histories under the same context require different action continuations, collapsing them prevents the action flow from fitting both; if they imply different physical consequences, the latent target must also distinguish them to predict the future. Histories that imply the same behavior and predictive consequence need not be separated. Joint prediction can therefore organize the representation around distinctions needed for control, as illustrated in Figure 5. Flow matching further retains differences beyond a single average action when valid continuations are multimodal. This is an emergent training pressure rather than a universal guarantee against collapse. Appendix F.1 gives the formal risk analysis, its scope, and the matched MSE comparison.

Objective Function. Recorded states and actions teacher-force the same history. The training objective is one likelihood for their joint continuation:

$$\mathcal{L}_{\text{joint}} = -\frac{1}{|\mathcal{B}|} \sum_{(\tau, t) \in \mathcal{B}} \log p_{\Theta}(\mathbf{z}_{t+1}, \mathbf{a}_{t+1} \mid \mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{c}_t), \quad \mathcal{L}_{\text{LeWAM}} = \mathcal{L}_{\text{joint}}. \quad (4)$$

The final realization parameterizes the action coordinate with conditional flow matching (Lipman et al., 2023); latent and action targets therefore shape the shared representation through one objective, without an auxiliary anti-collapse term. A fixed-variance Gaussian yields the matched MSE ablation, which preserves the same interface but estimates only a conditional mean. Appendix F.1 isolates this distinction, and Appendix C specifies both implementations.

Inference. Direct control reads the action marginal of the joint model,

$$p_{\Theta}(\mathbf{a}_{t+1} \mid \mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{c}_t) = \int p_{\Theta}(\mathbf{z}_{t+1}, \mathbf{a}_{t+1} \mid \mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{c}_t) d\mathbf{z}_{t+1}. \quad (5)$$

For goal-conditioned trajectories, this marginal amortizes the demonstrated mapping from history and goal to action. Without a goal, it is a continuation model of the behavior distribution. *Joint imagination* instead samples both outputs recursively:

$$(\hat{\mathbf{z}}_{t+k+1}, \hat{\mathbf{a}}_{t+k+1}) \sim p_{\Theta}(\mathbf{z}_{t+k+1}, \mathbf{a}_{t+k+1} \mid \hat{\mathbf{z}}_{\leq t+k}, \hat{\mathbf{a}}_{\leq t+k}, \mathbf{c}_t). \quad (6)$$

The action track proposes how behavior continues, and the latent track predicts the corresponding change. This rollout supplies the model and action generator used by our planner.

LeWAM Planning. The joint predictor already amortizes a history- and goal-conditioned distribution over coherent action continuations. When direct execution is insufficient, Steer-MPC performs test-time search through this frozen continuation model; no model parameter is updated. For action dimension d_a , raw-action planning exposes Hd_a coordinates and population methods repeatedly score samples in that space. We instead optimize a bounded steering variable whose dimension does not grow with H , while the frozen action conditional regenerates a complete action sequence after every update.

This design treats demonstrated behavior as reusable temporal structure. Play-LMP (Lynch et al., 2019) organizes demonstrations in a latent plan space, while VINN (Pari et al., 2021) retrieves nearby demonstrations in a learned visual representation. Steer-MPC performs neither latent-plan inference nor explicit dataset retrieval; every candidate is generated by the same history- and goal-conditioned action conditional. A steering update therefore changes the complete action chunk through correlations learned from data rather than assigning every time step an independent search coordinate. We call this *behavior-consistent search*: it biases candidates toward the learned continuation family without claiming that every generated action is strictly in distribution. **Steer-MPC searches by steering a learned continuation rather than constructing a plan action by action.** It freezes LeWAM and optimizes one *bounded steering vector* $\mathbf{v} \in \mathbb{R}^{d_v}$ added to the encoded goal $\mathbf{z}_g = \text{Enc}(\mathbf{o}_g)$:

$$\begin{aligned} \tilde{\mathbf{z}}_g(\mathbf{v}) &= \mathbf{z}_g + \mathbf{v}, & \mathbf{A}(\mathbf{v}) &= \pi_{\Theta}(\mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \tilde{\mathbf{z}}_g(\mathbf{v})), \\ \hat{\mathbf{z}}_{t+H}(\mathbf{v}) &= F_{\Theta}^{(H)}(\mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{A}(\mathbf{v})), & \mathbf{v}^* &= \arg \min_{\|\mathbf{v}\|_2 \leq \rho} d(\hat{\mathbf{z}}_{t+H}(\mathbf{v}), \mathbf{z}_g). \end{aligned} \quad (7)$$

Here π_{Θ} and F_{Θ} denote the action and latent-dynamics conditionals of the same frozen joint predictor, H is the planning horizon, and ρ is the trust-region radius. The search dimension d_v does

not grow with H , although evaluating each candidate still requires an H -step rollout; the benefit is a compact proposal parameterization, not horizon-independent rollout cost. The perturbed query $\mathbf{z}_g + \mathbf{v}$ generates a candidate behavior, whereas the unmodified $\mathbf{z}_g$ remains the evaluation target. In our realization, F_Θ is goal blind, so $\mathbf{v}$ can affect the rollout only through actions generated by π_Θ ; it cannot improve the objective by moving the target itself. The first action of $\mathbf{A}(\mathbf{v}^*)$ is executed before replanning. Search is thus confined to $\{\mathbf{A}(\mathbf{v}) : \|\mathbf{v}\|_2 \leq \rho\}$, a model-induced continuation family. This restriction reduces direct exposure to unsupported action coordinates, although it still requires calibrated dynamics, a meaningful latent goal cost, and a bounded trust region. Steer-MPC therefore changes the geometry of candidate generation; it does not claim to repair an inaccurate world model or an objective that misorders physical reachability. Appendix E.2 gives the algorithm and matched planner definitions.

Local continuation retrieval hypothesis. Viewed locally, we hypothesize that the frozen action conditional behaves as a goal-conditioned soft retriever or interpolator over demonstrated continuation chunks. This interpretation differs from explicit nonparametric retrieval in VINN (Pari et al., 2021): no dataset index participates in Steer-MPC inference, and every candidate is regenerated by the learned conditional model. For the query $\mathbf{q} = (\mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{z}_g)$, this interpretation writes

$$p_\Theta(\mathbf{A} \mid \mathbf{q}) \approx \sum_{i \in \mathcal{N}_k(\mathbf{q})} w_i(\mathbf{q}) \mathcal{K}_\sigma(\mathbf{A}; \mathbf{A}_i), \quad w_i(\mathbf{q}) \geq 0, \quad \sum_i w_i(\mathbf{q}) = 1, \quad (8)$$

where $\mathbf{A}_i$ is a demonstrated action continuation, $\mathcal{K}_\sigma$ is a local action kernel centered on that continuation, and $\mathcal{N}_k(\mathbf{q})$ denotes a local query neighborhood. Under this hypothesis, $\mathbf{v}$ is a continuous query edit: replacing $\mathbf{z}_g$ by $\mathbf{z}_g + \mathbf{v}$ changes which demonstrated continuations are locally emphasized, while the frozen action conditional still generates every complete candidate. Because the retrieval unit is a continuation rather than an isolated action, this hypothesis provides a concrete mechanism for retaining learned temporal dependencies during long-horizon search. Steer-MPC therefore searches for behavior-consistent candidates through the learned continuation model rather than searching the dataset explicitly or optimizing each action coordinate independently. Equation 8 is an explanatory, testable hypothesis, not a claim that the predictor implements literal k -nearest-neighbor inference.

Policy-mediated temporal coupling follows from sharing the same $\mathbf{v}$ across the entire horizon. Let $\mathbf{A}(\mathbf{v}) = [\mathbf{a}_{t+1}(\mathbf{v})^\top, \dots, \mathbf{a}_{t+H}(\mathbf{v})^\top]^\top$ and $\mathbf{J}_\mathbf{A}(\mathbf{v}) = \partial \mathbf{A}(\mathbf{v}) / \partial \mathbf{v}$. Locally,

$$\delta \mathbf{A} \approx \mathbf{J}_\mathbf{A}(\mathbf{v}) \delta \mathbf{v}, \quad \text{Cov}[\delta \mathbf{A}] \approx \mathbf{J}_\mathbf{A}(\mathbf{v}) \Sigma_v \mathbf{J}_\mathbf{A}(\mathbf{v})^\top, \quad \text{rank}(\text{Cov}[\delta \mathbf{A}]) \leq d_v, \quad (9)$$

for local steering perturbations with covariance Σ_v . The off-diagonal temporal blocks can be nonzero, so one steering update can change several time steps coherently instead of assigning each action an independent coordinate. The frozen, state-dependent action conditional therefore converts a low-dimensional steering update into a temporally coupled plan. Temporal coupling alone does not guarantee smooth or low-frequency actions; Section 4.6 tests the induced structure directly.

4 EXPERIMENTS

We evaluate LeWAM from controlled simulation to physical deployment: first across short- and long-horizon control, then through mechanism ablations, test-time planning, and real-robot tasks.

4.1 BENCHMARK: TOWARD GENERAL-PURPOSE JEPa CONTROL EVALUATION

To evaluate JEPa control across diverse dynamics, embodiments, and horizons, we use eight environments spanning navigation, planar control, and articulated, bimanual, dexterous, precision, contact-rich, and long-horizon tasks (Figure 6). Four follow LeWM (Maes et al., 2026b) task definitions; the remaining four add three long-horizon manipulation tasks and PointMaze, which uses the complex PointMaze-Large layout. ToolHang, Transport, and DrawerCleanup expose 7-, 14-, and 24-dimensional raw actions; PointMaze remains two-dimensional but requires long-range route composition. Appendix Table 2 reports these dimensions alongside benchmark-validity controls.

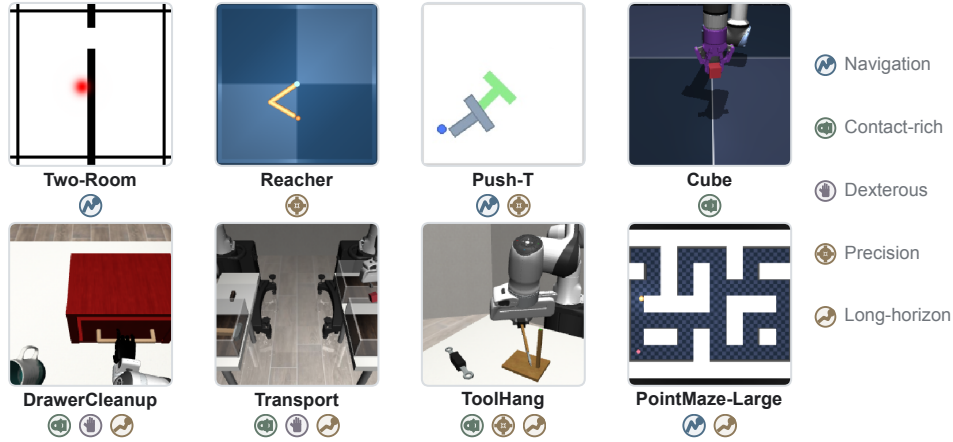


Figure 6: **Representative simulation environments.** The first row shows four environments used by LeWM; the second row introduces complementary navigation and manipulation settings. Symbols beneath each frame denote overlapping task attributes; long-horizon tasks require sustained decision-making over their native task durations.

Short-Horizon Goal Reaching. We evaluate the visual goal-reaching setting used by DINO-WM (Zhou et al., 2024) and LeWM (Maes et al., 2026b). Following the exact LeWM protocol, each episode samples an initial observation from an offline trajectory, takes the observation 25 steps later as the visual goal, and allows at most 50 environment actions.

Task Completion. Each episode uses the first observation of a complete trajectory as its initial condition and the terminal observation as its visual goal. The start-goal separation follows the native, task-dependent trajectory horizon, requiring the controller to complete the full recorded task rather than reach a nearby intermediate state.

Evaluation Metric. Within each protocol, all methods receive identical start-goal pairs, action limits, execution budgets, and task-specific success predicates. We report success rate over ten evaluation seeds with 50 episodes per seed unless stated otherwise. The main paper reports mean and standard deviation. Model and planner latency exclude simulator and rendering time. Exact task predicates and planner budgets appear in Appendix E.

Compared Methods. Colors identify the same model family throughout the paper.

- LeWAM reads the action conditional of the joint predictor in one forward pass.
- GC-IDM (Nguyen et al., 2026) freezes a pretrained LeWM and post-trains a goal-conditioned inverse model for direct control.
- LeWM (Maes et al., 2026b) learns SIGReg-regularized, action-conditioned latent dynamics and selects actions with CEM.
- DINO-WM (Zhou et al., 2024) learns dynamics over frozen DINOv2 features and selects actions with CEM. Main figures use its patch-token variant.
- GCBC (Ghosh et al., 2021) trains a goal-conditioned action predictor over frozen DINOv2 features.

4.2 RESULTS ON SHORT-HORIZON GOAL REACHING

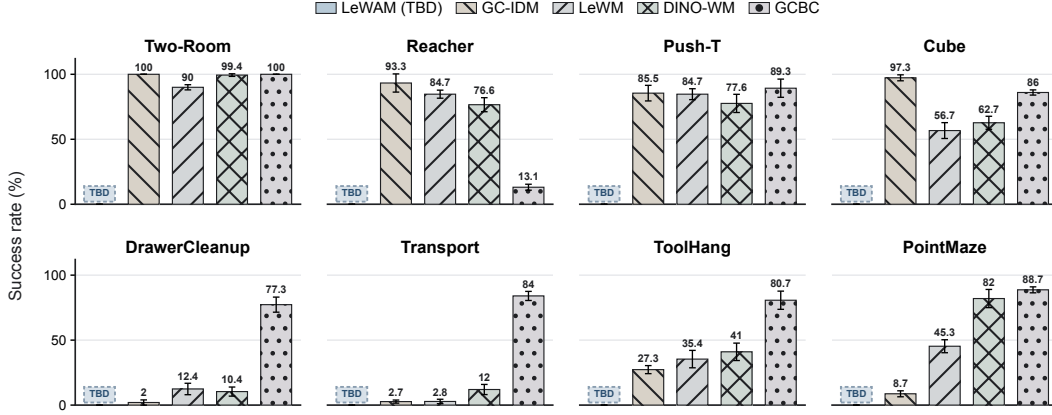


Figure 7: **Goal reaching.** Each panel denotes one environment under the same-trajectory, 25-step goal protocol and 50-action execution budget. Filled bars report current measurements; fixed-height dashed TBD boxes mark unreported entries and do not encode a success rate. Cube uses the corrected end-effector-aware (+EEF) predicate. Error bars denote one standard deviation.

4.3 RESULTS ON TASK COMPLETION

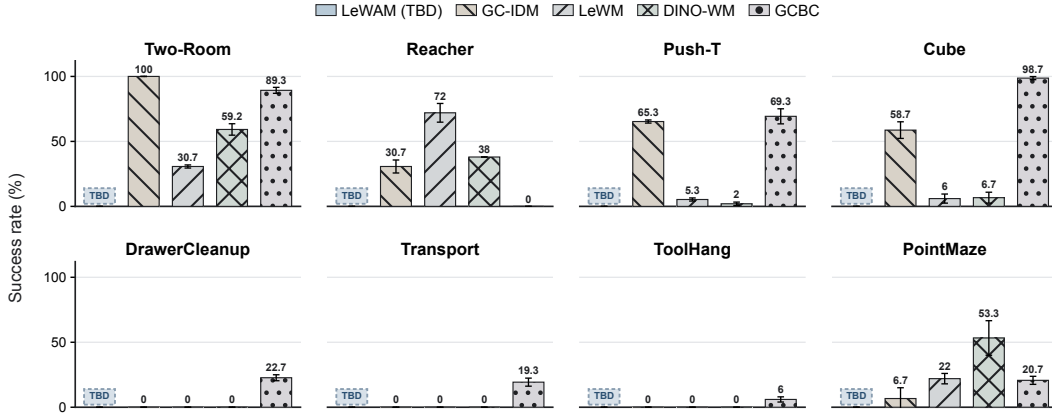


Figure 8: **Task completion.** Each panel denotes one environment with the first observation as the initial condition and the terminal observation of the complete trajectory as the goal. Fixed-height dashed TBD boxes mark unreported entries and do not encode a success rate. Error bars denote one standard deviation.

On Cube, GCBC performs better on task completion than on short-horizon goal reaching (98.7% vs. 86.0%), revealing a demonstration-completion bias in behavior cloning.

4.4 EMERGENCE OF CONTROL-CENTRIC PHYSICAL STRUCTURE

Structure from prediction alone. LeWAM introduces neither explicit state supervision nor an auxiliary representation regularizer. We therefore test whether joint prediction organizes its latent space around physical distinctions needed for control, rather than merely sustaining marginal variance. Frozen representations are evaluated with observed-state and contact-mode probes, future-displacement prediction, temporal ordering, and planning from imagined rollouts. Matched latent-

only and shuffled-action controls test whether the structure comes from sensorimotor pairing, while joint MSE tests whether modeling the full conditional action distribution matters beyond its mean. Controlled appearance and action-relevant geometry shifts further test whether the representation is selective for physical changes that alter control. Together, these probes test the claim that control-centric physical structure can emerge from the joint predictive objective without SIGReg. *Placeholder*.

4.5 WHY END-TO-END JOINT FLOW MATCHING?

Matched ablations. We test each design choice behind the one-stage objective: learning the visual encoder from scratch versus using frozen DINOv2 features; training with and without SIGReg; joint end-to-end prediction versus a frozen action-conditioned world model followed by a post-trained inverse model; and joint flow matching versus joint MSE. These controls separate visual pretraining, auxiliary anti-collapse, training stages, and target parameterization while matching data, capacity, and evaluation. Appendix F.1 specifies every comparison. *Placeholder*.

Table 1: **Learning the visual encoder end-to-end.** The frozen variants retain the joint LeWAM predictor but replace its from-scratch visual encoder with frozen DINOv2 CLS or patch features. We report task-completion success rates on the four manipulation environments. The from-scratch LeWAM row remains a placeholder until its matched runs are frozen.

Visual encoder	Task-completion success rate (%) $\uparrow$			
	Cube	Drawer	Transport	ToolHang
learned from scratch	—	—	—	—
frozen DINOv2 CLS	4.7 ± 3.1	21.3 ± 1.2	0.0 ± 0.0	4.0 ± 2.0
frozen DINOv2 patch	4.7 ± 3.1	20.0 ± 8.7	0.0 ± 0.0	2.0 ± 2.0

4.6 TOWARD SCALABLE PLANNING

Success—compute frontier. We compare direct LeWAM, Steer-MPC, CEM, iCEM, MPPI, and raw-action Gradient MPC using frozen checkpoints, identical starts and goals, matched horizons, and matched predictor-evaluation budgets. We report success against planner-only latency, end-to-end decision time, and model evaluations. *Placeholder*.

Search-coordinate ablation. We isolate the effect of steering coordinates by comparing one horizon-shared v , time-indexed steering vectors $v_{1:H}$, and free actions $a_{1:H}$ under matched initialization, rollout models, and optimization budgets. A zero-offset candidate and a random-offset control distinguish learned steering from extra gradient evaluations. *Placeholder*.

Temporal coupling and stability. We compare the action sequences induced by Steer-MPC with the white-noise proposal of CEM and the colored-noise proposal of iCEM. Action autocorrelation, power spectral density, total variation, and the cross-time blocks of Equation 9 quantify temporal structure; solver failure rate and sensitivity to initialization quantify refinement stability. *Placeholder*.

Local continuation retrieval. We test the hypothesis in Equation 8 by indexing demonstrated action chunks with frozen LeWAM query embeddings and measuring how steering changes nearest-neighbor identity, neighbor weights, and action-chunk distance to the demonstration set. A one-step retrieval control tests whether continuation-level retrieval better preserves cross-time action structure and terminal goal progress over long horizons. We jointly report realized goal progress, temporal consistency, and action-support distance for Steer-MPC, free-action Gradient MPC, random steering, and shuffled goal-continuation controls under matched rollout budgets. *Placeholder*.

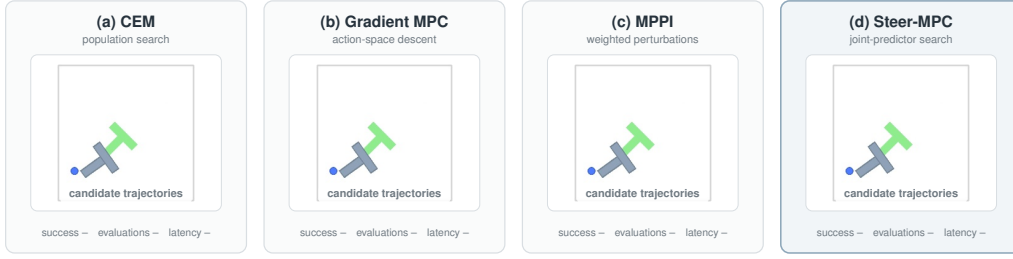


Figure 9: **Matched planner behavior.** Each panel overlays candidate trajectories and the selected solution for the same start, goal, frozen predictor, and planning horizon. Annotations report success, predictor evaluations, and planner-only latency.

4.7 RESULTS ON REAL ROBOTS

The original DINO-WM (Zhou et al., 2024), PLDM (Sobal et al., 2025), and LeWM (Maes et al., 2026b) studies report control in simulation rather than closed-loop physical-robot deployment. Our robot study asks whether this reconstruction-free predictive paradigm can transfer to physical manipulation by converting a video foundation model’s visual and temporal prior into controllable structure. We use a frozen V-JEPA 2.1 backbone (Mur-Labadia et al., 2026) and a trainable joint flow-matching predictor on a seven-DoF robot. Charging-block pick-and-place represents rigid-object manipulation; towel folding represents long-horizon deformable interaction.

The primary factorial study crosses backbone initialization (pretrained or identically frozen random) with prediction target (actions only or future representations and actions). All four cells share robot data, Transformer capacity, optimization, action chunks, and closed-loop execution. We report success with raw trial outcomes, data efficiency by recorded frames or hours, held-out visual and object shifts, and model-only and network-inclusive latency. Appendix G gives the complete realization. *Placeholder.*

5 LIMITATIONS AND SCOPE

LeWAM learns control-centric structure from demonstrations, so its invariances are relative to the demonstrated behavior distribution. Visual distinctions that change neither the demonstrated action continuation nor its predicted physical consequence may be compressed; transfer to unseen tasks that require those distinctions may therefore require broader interaction data or adaptation. Moreover, action-dependent separation is a training pressure rather than an unconditional guarantee against partial collapse or contextual shortcuts. We accordingly ground the representation claim in physical probes, controlled shifts, direct control, and imagined planning within the evaluated task scope.

6 CONCLUSION

We introduced **LeWAM**, a succinct JEPA that models latent-state and action continuations through one joint predictive objective. Action prediction identifies how demonstrated behavior is realized, while latent prediction grounds that behavior in its physical consequence; together they organize the representation around distinctions needed for control without an explicit anti-collapse regularizer. The same frozen predictor supports direct control and Steer-MPC, which refines a bounded goal-latent steering variable instead of searching over an unconstrained action sequence. Across simulated and physical-robot tasks, the resulting system addresses a broader control scope than prior JEPA benchmarks. More broadly, the results support a task-aligned view of representation learning: useful physical structure can emerge from the control objective itself.

REFERENCES

Pulkit Agrawal, Ashvin Nair, Pieter Abbeel, Jitendra Malik, and Sergey Levine. Learning to poke by poking: Experiential learning of intuitive physics. In *Advances in Neural Information Processing*

- Systems (NeurIPS)*, 2016.
- Brandon Amos. Tutorial on amortized optimization. *arXiv preprint arXiv:2202.00665*, 2022.
- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.
- Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022.
- Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.
- Kevin Black, Mitsuhiko Nakamoto, Pranav Atreya, Homer Walke, Chelsea Finn, Aviral Kumar, and Sergey Levine. Zero-shot robotic manipulation with pretrained image-editing diffusion models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, et al. Genie: Generative interactive environments. In *International Conference on Machine Learning (ICML)*, 2024.
- Samuel A. Burden, S. Shankar Sastry, Daniel E. Koditschek, and Shai Revzen. Event-selected vector field discontinuities yield piecewise-differentiable flows. *SIAM Journal on Applied Dynamical Systems*, 15(2):1227–1267, 2016.
- Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Quentin Gallouédec, Thomas Wolf, et al. LeRobot: An open-source library for end-to-end robot learning. *arXiv preprint arXiv:2602.22818*, 2026.
- Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, et al. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.
- Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023.
- Matthieu Destrade, Oumayma Bounou, Quentin Le Lidec, Jean Ponce, and Yann LeCun. Value-guided action planning with JEPA world models. *arXiv preprint arXiv:2601.00844*, 2026.
- Dibya Ghosh, Abhishek Gupta, Ashwin Reddy, Justin Fu, Coline Devin, Benjamin Eysenbach, and Sergey Levine. Learning to reach goals via iterated supervised learning. In *International Conference on Learning Representations (ICLR)*, 2021.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- Kristian Hartikainen, Xinyang Geng, Tuomas Haarnoja, and Sergey Levine. Dynamical distance learning for semi-supervised and unsupervised skill discovery. In *International Conference on Learning Representations (ICLR)*, 2020.
- Mikael Henaff, William F. Whitney, and Yann LeCun. Model-based planning with discrete and continuous actions. *arXiv preprint arXiv:1705.07177*, 2017.
- Petr Ivashkov, Randall Balestriero, and Bernhard Schölkopf. Sensorimotor world models: Perception for action via inverse dynamics. *arXiv preprint arXiv:2606.20104*, 2026.
- S V Jyothir, Siddhartha Jalagam, Yann LeCun, and Vlad Sobal. Gradient-based planning with world models. *arXiv preprint arXiv:2312.17227*, 2023.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Nathan J. Kong, J. Joe Payne, James Zhu, and Aaron M. Johnson. Saltation matrices: The essential tool for linearizing hybrid dynamical systems. *Proceedings of the IEEE*, 112(6):585–608, 2024.
- Lukas Kuhn, Lucas Maes, Giuseppe Serra, Quentin Le Lidec, Yann LeCun, Randall Balestriero, and Florian Buettner. LeVJEPa: Efficient & scalable video pretraining without the heuristics. *arXiv preprint arXiv:2608.27395*, 2026.
- Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. *arXiv preprint arXiv:2503.00200*, 2025.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- Hao Luo, Wanpeng Zhang, Yicheng Feng, Sipeng Zheng, Haiweng Xu, Chaoyi Xu, Ziheng Xi, Yuhui Fu, and Zongqing Lu. Being-H0.7: A latent world-action model from egocentric videos. *arXiv preprint arXiv:2605.00078*, 2026.
- Corey Lynch, Mohi Khansari, Ted Xiao, Vikash Kumar, Jonathan Tompson, Sergey Levine, and Pierre Sermanet. Learning latent plans from play. In *Conference on Robot Learning (CoRL)*, 2019.
- Xindian Ma, Wen Yuan Liu, Peng Zhang, and Nan Xu. 3D-RPE: Enhancing long-context modeling through 3d rotary position encoding. *arXiv preprint arXiv:2406.09897*, 2024.
- Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation. *arXiv preprint arXiv:2605.21800*, 2026a.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026b.
- Sergi Masip, Jonathan Swinnen, Yutong Hu, Renaud Detry, and Tinne Tuytelaars. FF-JEPa: Long-horizon planning in world models with latent planners. *arXiv preprint arXiv:2606.09311*, 2026.
- Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Lorenzo Mur-Labadia, Matthew Muckley, Amir Bar, Mahmoud Assran, Koustuv Sinha, Michael Rabbat, Yann LeCun, Nicolas Ballas, and Adrien Bardes. V-JEPa 2.1: Unlocking dense features in video self-supervised learning. *arXiv preprint arXiv:2603.14482*, 2026.
- Hoang Nguyen, Xiaohao Xu, and Xiaonan Huang. Latent geometry beyond search: Amortizing planning in world models. *arXiv preprint arXiv:2605.08732*, 2026.

- Tao Pang, H. J. Terry Suh, Lujie Yang, and Russ Tedrake. Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models. *IEEE Transactions on Robotics*, 39(6):4691–4711, 2023. doi: 10.1109/TRO.2023.3300230.
- Jyothish Pari, Nur Muhammad Shafiullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The surprising effectiveness of representation learning for visual imitation. *arXiv preprint arXiv:2112.01511*, 2021.
- Arjun Parthasarathy, Nimit Kalra, Rohun Agrawal, Yann LeCun, Oumayma Bounou, Pavel Izmailov, and Micah Goldblum. Closing the train-test gap in world models for gradient-based planning. *arXiv preprint arXiv:2512.09929*, 2025.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Cristina Pinneri, Shambhuraj Sawant, Sebastian Blaes, Jan Achterhold, Joerg Stueckler, Michal Rolinek, and Georg Martius. Sample-efficient cross-entropy method for real-time planning. In *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pp. 1049–1065. PMLR, 2021. URL <https://proceedings.mlr.press/v155/pinneri21a.html>.
- Michael Posa, Cecilia Cantu, and Russ Tedrake. A direct method for trajectory optimization of rigid bodies through contact. *The International Journal of Robotics Research*, 33(1):69–81, 2014.
- Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1:127–190, 1999.
- Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv preprint arXiv:2502.14819*, 2025.
- H. J. Terry Suh, Tao Pang, and Russ Tedrake. Bundled gradients through contact via randomized smoothing. *IEEE Robotics and Automation Letters*, 7(2), 2022.
- Stephen Tian, Suraj Nair, Frederik Ebert, Sudeep Dasari, Benjamin Eysenbach, Chelsea Finn, and Sergey Levine. Model-based visual planning with self-supervised functional distances. In *International Conference on Learning Representations (ICLR)*, 2021.
- Junke Wang, Qihang Zhang, Shuai Yang, Yiming Luo, Yujun Shen, Zuxuan Wu, Yu-Gang Jiang, and Yinghao Xu. RepWAM: World action modeling with representation visual-action tokenizers. *arXiv preprint arXiv:2606.13674*, 2026a.
- Kaixin Wang, Kuangqi Zhou, Jiashi Feng, Bryan Hooi, and Xinchao Wang. Reachability-aware laplacian representation in reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2023.
- Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. *arXiv preprint arXiv:2603.12231*, 2026b.
- Yuhai Wang, Jiawei Xia, Rongxuan Zhou, Xiao Hu, Yongliang Shi, Jing Du, and Yang Ye. PRISM: PRior-guided Imagination Sampling in world Models. *arXiv preprint arXiv:2606.07974*, 2026c.
- Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic MPC for model-based reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- Haiyu Wu, Randall Balestriero, and Morgan Levine. VISReg: Variance-invariance-sketching regularization for JEPA training. *arXiv preprint arXiv:2606.02572*, 2026.
- Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.

- Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.
- Wancong Zhang, Basile Terver, Artem Zhohus, Soham Chitnis, Harsh Sutaria, Mido Assran, et al. Hierarchical planning with latent world models. *arXiv preprint arXiv:2604.03208*, 2026a.
- Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong, Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, Chenxi Bao, Tao Yu, Jingjing Zhou, and Jungang Xu. Delta-JEPA: Learning action-sensitive world models via latent difference decoding. *arXiv preprint arXiv:2606.31232*, 2026b.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.
- Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.

Appendix

Supplementary material for **LeWAM: JEPA Can be Succinct and Realistic**

Roadmap

A	Reproducibility, Ethics, and Safety	p. 17
B	Questions and Answers	p. 17
C	Implementation Details	p. 18
	Baseline Implementations	p. 18
	System-Level Simplicity and Cost	p. 19
D	Datasets and Benchmark Construction	p. 19
	Benchmarks and Tasks	p. 19
	Construction and Validation	p. 19
E	Evaluation Protocol	p. 21
	Goals and Success Criteria	p. 21
	Planning Algorithm Comparison	p. 22
	Runtime and Cost Alignment	p. 24
F	Additional Analyses and Numerical Results	p. 25
	Why Joint Flow Matching?	p. 25
	Why End-to-End Joint Prediction?	p. 26
	Behavior Cloning	p. 27
	Contact-Sensitive Latent Planning	p. 27
	Robustness and Representation	p. 27
	Qualitative Results and Failure Cases	p. 28
G	Real-Robot System: Architecture, Platform, and Data	p. 29
H	Recommendations for Future Work	p. 34

A REPRODUCIBILITY, ETHICS, AND SAFETY

Reproducibility statement. Our machine-readable ledger traces each result to its protocol, seeds, run identifier, checkpoint, and aggregation rule; a companion log records protocol changes and failed or superseded runs. The main paper reports aggregate statistics, whereas Appendix F preserves the underlying value from every retained local run. We distinguish reported values from local reproductions and never reconstruct seed-level observations from a published mean and standard deviation. The artifact will release code, configurations, split and goal-generation scripts, result records, seeds, and redistributable checkpoints. Sections C–E document the remaining details.

Ethics and asset licensing statement. The experiments collect no new human-subject data. They reuse environments, datasets, models, and software governed by their original licenses. The artifact manifest will record source, version, license, modifications, citation, and redistribution status for each asset. Restricted materials will be distributed as acquisition and preprocessing instructions. Learned action models may fail under distribution shift and require independent safety mechanisms before physical deployment.

B QUESTIONS AND ANSWERS

Q1: On Search-Free JEPA. *Is search-free control the contribution of LeWAM?*

No. Policies such as Diffusion Policy (Chi et al., 2023), inverse-dynamics models such as GC-IDM (Nguyen et al., 2026), and world action models such as DreamZero (Ye et al., 2026) already emit actions without online trajectory search. Direct execution is one inference mode of LeWAM, not its novelty boundary. The contribution is a single JEPA predictor whose deployed outputs include action-conditioned World Prediction, an action marginal, and coupled state–action imagination. Search can be omitted when the learned action conditional is sufficient or retained through Steer-MPC when test-time refinement is useful.

Q2: On Behavior Cloning. *Is LeWAM goal-conditioned behavior cloning with a world-model auxiliary loss?*

Its action marginal answers the same conditional query as goal-conditioned behavior cloning, so direct success alone cannot distinguish the two. The stronger claim requires the other conditional to remain operational: candidate actions must change predicted futures, imagined futures must affect action probabilities, and joint rollouts must support planning. The latent-only, action-only, split, and joint ablations in Section 4.5 test these dependencies under matched capacity. If the action branch ignores the latent prediction target, the result should be described as a policy with an auxiliary loss rather than as a world action model.

Q3: On Goal Conditioning. *What role does it play, and what if no goal image is available?*

Goal conditioning selects a task-directed continuation. The compact planning experiments use a goal image, while the current single-task robot models omit c_t and learn the continuation distribution of each task directly. Other conditioning modalities fit the same interface but are not evaluated here. A language embedding can follow a vision-language-action model such as OpenVLA (Kim et al., 2024); a generated goal observation can follow SuSIE (Black et al., 2024), which synthesizes a future image from the current observation and instruction before invoking a goal-conditioned controller.

Q4: On Collapse Prevention. *Why does LeWAM omit an explicit anti-collapse regularizer?*

The joint-flow realization of LeWAM adds no auxiliary anti-collapse mechanism. Instead, action-relevant representation separation can emerge from the intended likelihood: when visual history changes the conditional action flow, a constant visual code cannot attain its optimal risk. This property is not generic to every action head; MSE can discard distributional differences that do not change the conditional mean. Appendix F.1 therefore compares joint flow with joint MSE without SIGReg, then evaluates joint versus latent-only MSE with SIGReg and parameter count matched.

Q5: On the Steer-MPC Search Variable. *Why optimize a goal-latent offset instead of the action sequence?*

Raw-action Gradient MPC (Henaff et al., 2017; Jyothir et al., 2023; Parthasarathy et al., 2025) can move into action regions where a next-state predictor has not been trained, producing low model cost without reliable physical progress. Steer-MPC searches by steering a continuation generated by the frozen action conditional under bounded goal perturbations. The dynamics rollout is goal blind and the cost retains the true goal, so the offset can improve the score only by changing the policy-generated actions. This removes free action coordinates and direct goal leakage; it does not prove that every generated plan lies on the empirical data manifold or that the world model cannot be exploited. The trust-region, support-distance, and predicted-versus-realized progress studies in Section 4.6 test that narrower claim.

C IMPLEMENTATION DETAILS

Inner architecture. *Placeholder.*

Optimization. *Placeholder.*

C.1 BASELINE IMPLEMENTATIONS

We implement all baselines in the released Stable World Model codebase (Maes et al., 2026a), retaining its environment wrappers, dataset interface, action bounds, and evaluation loop. Unless stated below, each method follows its official implementation and hyperparameters.¹ Within each protocol, all methods use the same observations, actions, start-goal pairs, and success predicates. We list only deviations and protocol choices that are not evident from the released configurations.

- **GC-IDM.** We follow the official two-stage implementation of Nguyen et al. (2026). Goal reaching samples a remaining horizon uniformly from 1 to 50 and clips it to the episode boundary. Task completion uses the final image of the same trajectory as its only goal input and omits normalized-horizon conditioning. The two protocol-specific heads share one frozen latent cache. Compared with the official code, we move random goal sampling from minibatch loading to an equivalent precomputation stage, preserving the sampling distribution while reducing data-loading overhead.
- **LeWM.** We follow the official LeWM training and CEM evaluation settings (Maes et al., 2026b). On the original four goal-reaching benchmarks, our reproduced checkpoints perform similarly to the released checkpoints; all reported results use our checkpoints. Goal reaching and task completion differ only in goal construction and evaluation horizon, and each protocol is evaluated over ten seeds.
- **DINO-WM.** We evaluate the released patch-token model and a matched CLS-token variant; tokenization is their only architectural difference. The four added tasks use ten training epochs, while the original four retain their four-epoch checkpoints. One checkpoint supports both protocols. Task completion executes 25 environment steps between CEM replans.
- **GCBC.** We use the goal-conditioned behavior-cloning baseline from the released codebase (Maes et al., 2026a), trained for 20 rather than 100 epochs. Goal reaching samples a uniformly later frame from the same episode. Task completion is trained separately and uses the final image of the same trajectory as its only goal input. At evaluation, each policy executes its predicted five-step action chunks without planning.

¹Official repositories: DINO-WM, Stable World Model, and GC-IDM.

C.2 SYSTEM-LEVEL SIMPLICITY AND COST

D DATASETS AND BENCHMARK CONSTRUCTION

D.1 BENCHMARKS AND TASKS

Short-horizon goal reaching. The initial state is sampled from an offline trajectory, and the state 25 steps later in the same trajectory serves as the visual goal. Each controller receives 50 environment actions. We use the same start-goal offset and execution budget across all eight environments. On `Two-Room`, `Reacher`, `Push-T`, and `Cube`, we retain the original task success predicates and report SR_1 for direct comparison with prior results.

Full-trajectory task-completion track. For every evaluation trajectory, the first observation is the initial condition and the final observation is the visual goal. This preserves demonstrated feasibility while replacing a nearby hindsight target with the complete recorded task horizon. Starts, goals, action limits, and success predicates are shared across methods; only the controller changes.

Diagnostic track. No training trajectory contributes an evaluation endpoint. Where the simulator exposes goal generation, start and goal states are sampled from held-out feasible configurations rather than coupled by one recorded continuation. Goal distance is stratified into near, medium, and long-horizon bands, with the final band extending beyond the hindsight window used for training.

Expanded diagnostic suite. The suite contains the four revised LeWM tasks in Figure 6 together with `DrawerCleanup`, `Transport`, `ToolHang`, and `PointMaze`. `DrawerCleanup` exercises dexterous hands, an articulated drawer, and relational object placement. `Transport` requires coordinated two-arm payload transfer. `ToolHang` tests precise contact and placement under a tight tolerance. `PointMaze` requires obstacle-aware route composition over a longer horizon. The resulting eight tasks expose the same observation-action trajectory interface and matched direct, latent-planning, and joint-imagination evaluation.

D.2 CONSTRUCTION AND VALIDATION

Dataset-paired goals. The released LeWM evaluation samples an initial state from an offline trajectory and a goal several steps later in that same trajectory (Maes et al., 2026b). This construction guarantees a demonstrated path to the goal, but it measures interpolation within the dataset and does not isolate planning to held-out routes, goals, or environment configurations. The diagnostic suite retains the released protocol as a compatibility track and introduces a trajectory-disjoint track for generalization.

Transient success. The released LeWM evaluator records an absorbing first hit: it accumulates environment termination after every step and freezes the rollout once the task predicate is satisfied. The original DINO-WM (Zhou et al., 2024) evaluator checks the endpoint of each executed planning segment and likewise accepts a single checked state. A controller that briefly satisfies the goal and then drifts away therefore scores exactly as one that arrives and holds. The diagnostic track replaces the single check with a K -step persistence criterion. Let $r_t^{(i)}$ be the binary composite goal predicate at step t of episode i . Over valid K -step windows, define

$$s_K^{(i)} = \mathbb{1}[\exists t : r_t^{(i)} = \dots = r_{t+K-1}^{(i)} = 1], \quad SR_K = \frac{1}{N} \sum_{i=1}^N s_K^{(i)}. \quad (10)$$

For LeWM, SR_1 exactly recovers the released metric, while $K > 1$ withholds credit until the predicate holds for consecutive steps. Persistent occupancy is the behavioral signature of goal-directed stabilization; a transient pass is not.

Partial predicates. A success predicate that reads fewer variables than the goal image can credit a state the goal never described. The `Cube` predicate checks cube translation alone, so object orientation and robot configuration visible in the goal image leave success unchanged. The diagnostic track scores task-complete predicates over every goal-relevant entity, which makes the metric answer the question the visual goal poses.

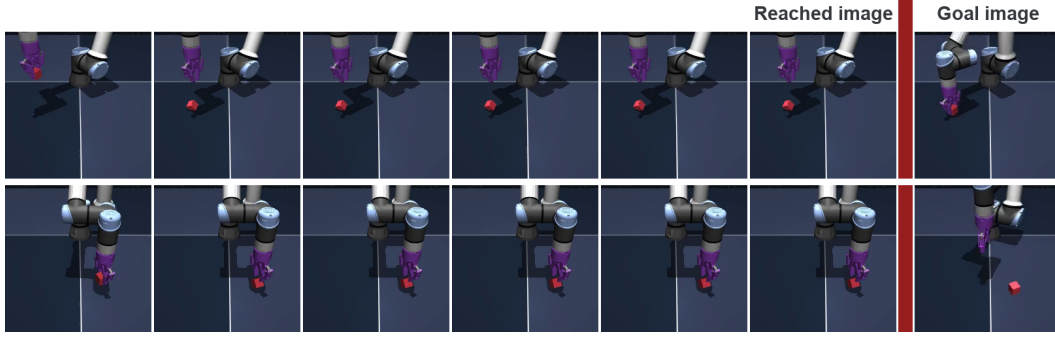


Figure 10: **A degenerate Cube success under short-horizon goal reaching.** Each row shows a policy rollout (left, frames over time) beside its goal frame (far right, after the divider). Throughout the execution the gripper never reaches the goal end-effector pose, yet the cube stays within the 0.04 m object-only threshold of the goal, so OGBench’s object-position-only predicate scores the episode a success. Our end-effector term (Table 2) rejects these degenerate successes by additionally requiring the gripper to reach the goal pose.

Task coverage. The suite revises the four LeWM tasks and adds four environments with complementary control structure. The additions target articulated, dexterous, bimanual, and precision manipulation together with obstacle-aware long-horizon navigation.

Evaluation contract. For the revised tasks, the compatibility track reproduces the released same-trajectory sampling and SR_1 . The primary diagnostic track holds evaluation trajectories out from training, separates held-out environment configurations, stratifies goals by temporal distance, and includes horizons beyond the training window. Starts that already satisfy the goal are removed from policy success and reported as a free-start diagnostic. Each task reports SR_K under its complete predicate together with random-action and demonstration-replay controls.

Task selection. *Placeholder.*

Environment and asset construction. *Placeholder.*

Split and goal construction. *Placeholder.*

Predicate and persistence design. A worked case makes the partial-predicate problem concrete. Cube inherits OGBench’s success test, which reads the cube center-of-mass alone: an episode succeeds when the cube lies within 0.04 m of its target, independent of the gripper. Under short-horizon goal reaching, where the goal is the demonstration frame `goal_offset_steps` = 25 ahead, the cube has often barely moved by the goal, so a policy that does nothing already satisfies the predicate. The object-only predicate is already satisfied at the initial state on roughly 43% of episodes, which is exactly the do-nothing floor reported in Table 2. We correct the predicate by adding an end-effector term,

$$\text{success} = (\|c_{\text{obj}} - c_{\text{obj}}^g\| < \tau_{\text{obj}}) \wedge (\|e_{\text{eff}} - e_{\text{eff}}^g\| < \tau_{\text{eff}}), \quad \tau_{\text{obj}} = \tau_{\text{eff}} = 0.04 \text{ m}, \quad (11)$$

requiring the gripper to also reach the goal-frame end-effector pose. The term is opt-in through an environment threshold that defaults to $+\infty$, so the predicate is byte-identical to OGBench’s unless enabled, and no other task is affected. It rejects the do-nothing successes: the Cube and Cube (+EEF) rows of Table 2 report the measured chance floor from zero-action and random policies, which the end-effector term cuts from roughly 43 to 3. A Cube success rate under the corrected predicate therefore reflects task-directed behavior rather than a near-satisfied initial state.

Controls and validation. *Placeholder.*

ENVIRONMENT	RAW ACTION DIM.	GOAL REACHING		TASK COMPLETION	
		ZERO-ACTION	RANDOM	ZERO-ACTION	RANDOM
Two-Room	2	7.3 $\pm$ 0.9	25.3 $\pm$ 5.0	0.0 $\pm$ 0.0	2.7 $\pm$ 2.5
Reacher	2	3.3 $\pm$ 1.9	12.0 $\pm$ 1.6	0.0 $\pm$ 0.0	12.0 $\pm$ 4.9
Push-T	2	0.0 $\pm$ 0.0	1.3 $\pm$ 0.9	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
Cube	5	42.7 $\pm$ 6.6	44.0 $\pm$ 1.6		
Cube (+EEF)	5	2.7 $\pm$ 0.9	4.7 $\pm$ 0.9	4.7 $\pm$ 2.5	6.0 $\pm$ 1.6
DrawerCleanup	24	0.0 $\pm$ 0.0	6.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
Transport	14	3.3 $\pm$ 0.9	2.7 $\pm$ 0.9	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
ToolHang	7	22.7 $\pm$ 2.5	26.0 $\pm$ 1.6	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
PointMaze	2	6.0 $\pm$ 0.0	9.3 $\pm$ 1.9	0.7 $\pm$ 0.9	6.0 $\pm$ 3.3

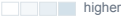
control floor  higher

Table 2: **Action dimensions and policy-floor controls.** Rows follow Figure 6, reading its first row and then its second row; Cube (+EEF) isolates the corrected end-effector predicate. Raw action dimensionality is reported beside zero-action and random-policy success rates (%) for goal reaching (50 steps) and task completion, averaged over three seeds. Because the EEF term changes only goal-reaching evaluation, task-completion controls are reported once in the Cube (+EEF) row. DrawerCleanup exposes 24 raw coordinates; the policy predicts 22 active dimensions after removing two constant joints. PointMaze uses the PointMaze-Large layout.

E EVALUATION PROTOCOL

E.1 GOALS AND SUCCESS CRITERIA

Backward-compatible evaluation. The compatibility track uses the released same-trajectory start-goal construction and first-hit success. It exists for direct comparison with published LeWM-benchmark numbers.

Generalization evaluation. The primary track separates training and evaluation trajectories, records held-out environment configurations as a distinct axis, and never credits an episode whose initial state already satisfies the goal. Feasible goals come from environment-native sampling when available and otherwise from the held-out evaluation split. Goal distance is reported by horizon band rather than averaged into a single near-goal score.

Composite success. Each manipulation predicate evaluates all state variables required by task completion, such as object pose, articulated-object state, and relevant end-effector relations. This is stricter than the released Cube predicate, which checks cube translation within 0.04 m but ignores cube orientation and robot configuration. We log every predicate component so aggregate success can be audited.

Protocol controls. Every task reports the free-start rate, a random-action floor, and a recorded-action replay control. Replay tests whether the sampled goal is reachable under the stored continuation and exposes simulator-version or state-restoration drift before model comparisons are interpreted.

Code-path audit. The public LeWM entry point delegates evaluation to `stable-worldmodel`. Its dataset evaluator applies a sticky logical OR between per-step environment termination and episode success; its wait mode then freezes a terminated rollout. Two-Room and Push-T terminate on threshold entry, Cube enables goal termination, and the Reacher qpos-matching task returns a terminal discount when every joint enters tolerance. In Reacher, the subsequent `step.last()` check therefore denotes early goal termination, not terminal-only scoring. The released LeWM metric is consequently first-hit success on all four tasks. By contrast, the original DINO-WM evaluator

extracts the final state of an executed action segment before applying the task predicate; its MPC wrapper retains these endpoint successes across replanning rounds.

Persistent evaluator. For a goal-reaching task, the environment records r_t and maintains a consecutive-hit counter. The counter increments while $r_t = 1$ and resets to zero whenever the trajectory leaves the region. Goal termination is suppressed until the counter reaches K , while the original time limit remains unchanged. We retain SR_1 for exact comparison with released LeWM results and report a persistence sweep separately. Goal thresholds, values of K , and complete predicate definitions are reported with each task.

E.2 PLANNING ALGORITHM COMPARISON

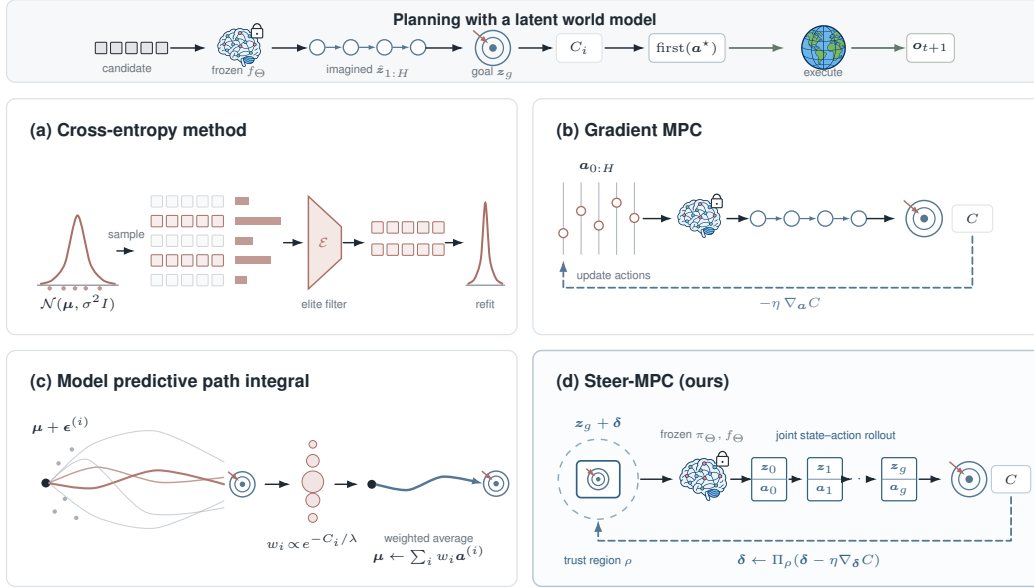


Figure 11: **Planning with latent dynamics.** All planners share the predictive model, goal cost, and receding-horizon execution, but differ in search variable and optimizer. Steer-MPC searches through the frozen action conditional; the alternatives optimize raw action sequences.

Setting and Notations. All planners use the same frozen encoder, latent dynamics F_Θ , terminal cost, and replanning schedule; Steer-MPC additionally uses the frozen action conditional π_Θ . Let z_t and z_g denote the current and goal latents, H the horizon, and $a_{0:H}$ a candidate action sequence. Its rollout defines

$$C(a_{0:H}) = \|\hat{z}_H(a_{0:H}) - z_g\|_2^2, \quad (12)$$

The algorithms below share N candidates, J iterations, and the symbols defined in their respective inputs. MPC executes the first action chunk and replans from the next observation.

Table 3: Planning algorithms and representative methods that instantiate them.

Planning algorithm	Representative methods
CEM	PlaNet (Hafner et al., 2019), DINO-WM (Zhou et al., 2024), LeWM (Maes et al., 2026b), V-JEPA 2-AC (Assran et al., 2025), Delta-JEPA (Zhang et al., 2026b)
iCEM	iCEM (Pinneri et al., 2021)
Gradient MPC	Differentiable forward-model planning (Henaff et al., 2017), DINO-WM (Zhou et al., 2024), Gradient-based Planning with World Models (Jyothir et al., 2023), Stable World Models (Maes et al., 2026a)
MPPI	MPPI (Williams et al., 2017), PLDM (Sobal et al., 2025), PRISM (Wang et al., 2026c), TD-MPC2 (Hansen et al., 2024)
Steer-MPC	LeWAM

Algorithm 1 Cross-entropy method

Require: z_t, z_g, H, N, N_e, J

- 1: $\mu \leftarrow \mathbf{0}, \quad \sigma \leftarrow \sigma_0$
- 2: **for** $j = 1, \dots, J$ **do**
- 3: sample $\mathbf{a}_{0:H}^{(i)} \sim \mathcal{N}(\mu, \sigma^2 I)$ for $i = 1, \dots, N$
- 4: $c_i \leftarrow C(\mathbf{a}_{0:H}^{(i)})$ by rolling each candidate through f_Θ
- 5: $\mathcal{E} \leftarrow$ indices of the N_e smallest c_i
- 6: $\mu, \sigma \leftarrow$ mean and standard deviation of $\{\mathbf{a}_{0:H}^{(i)}\}_{i \in \mathcal{E}}$
- 7: **end for**
- 8: **return** μ

iCEM. iCEM (Pinneri et al., 2021) uses the same objective and update as CEM, augmented with colored-noise sampling, elite reuse, and MPC shifting.

Algorithm 2 Gradient descent, the solver inside Gradient MPC

Require: z_t, z_g, H, N, J, η

- 1: sample $\mathbf{a}_{0:H}^{(i)} \sim \mathcal{N}(\mathbf{0}, \sigma_0^2 I)$ for $i = 1, \dots, N$ $\triangleright$ parallel restarts
- 2: **for** $j = 1, \dots, J$ **do**
- 3: **for** $i = 1, \dots, N$ **do**
- 4: $c_i \leftarrow C(\mathbf{a}_{0:H}^{(i)})$
- 5: $\mathbf{a}_h^{(i)} \leftarrow \mathbf{a}_h^{(i)} - \eta \partial c_i / \partial \mathbf{a}_h^{(i)}$ for $h = 0, \dots, H$ $\triangleright$ backpropagate through the rollout
- 6: **end for**
- 7: **end for**
- 8: **return** $\mathbf{a}_{0:H}^{(i^*)}$ with $i^* = \arg \min_i c_i$

Algorithm 3 Model predictive path integral control

Require: $z_t, z_g, H, N, J, \lambda$

- 1: $\mu \leftarrow \mathbf{0}$
- 2: **for** $j = 1, \dots, J$ **do**
- 3: sample $\epsilon^{(i)} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I)$ and set $\mathbf{a}_{0:H}^{(i)} \leftarrow \mu + \epsilon^{(i)}$
- 4: $c_i \leftarrow C(\mathbf{a}_{0:H}^{(i)})$
- 5: $w_i \leftarrow \exp(-c_i/\lambda) / \sum_{l=1}^N \exp(-c_l/\lambda)$
- 6: $\mu \leftarrow \sum_{i=1}^N w_i \mathbf{a}_{0:H}^{(i)}$ $\triangleright$ every sample contributes
- 7: **end for**
- 8: **return** μ

Algorithm 4 Steer-MPC

Require: $\mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{z}_g, H, J, \eta, \rho$

```

1:  $\mathbf{v} \leftarrow \mathbf{0}$ 
2:  $\mathbf{A}^* \leftarrow \pi_{\Theta}(\mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{z}_g)$ 
3:  $c^* \leftarrow C(\mathbf{A}^*)$  ▷ retain the unmodified action conditional
4: for  $j = 1, \dots, J$  do
5:    $\mathbf{A} \leftarrow \pi_{\Theta}(\mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{z}_g + \mathbf{v})$  ▷ frozen action conditional
6:    $\hat{\mathbf{z}}_{t+H} \leftarrow F_{\Theta}^{(H)}(\mathbf{z}_{\leq t}, \mathbf{a}_{\leq t}, \mathbf{A})$  ▷ goal-blind rollout
7:    $c \leftarrow d(\hat{\mathbf{z}}_{t+H}, \mathbf{z}_g)$  ▷ unmodified goal
8:   if  $c < c^*$  then
9:      $\mathbf{A}^* \leftarrow \mathbf{A}, \quad c^* \leftarrow c$ 
10:  end if
11:   $\mathbf{v} \leftarrow \mathbf{v} - \eta \partial c / \partial \mathbf{v}$ 
12:   $\mathbf{v} \leftarrow \mathbf{v} \cdot \min(1, \rho / (\|\mathbf{v}\|_2 + \epsilon))$  ▷ project onto  $\|\mathbf{v}\|_2 \leq \rho$ 
13: end for
14: return  $\mathbf{A}^*$ 

```

Only $\mathbf{v}$ carries a gradient; all model parameters remain frozen, and the retained zero-offset candidate prevents degradation under the model cost.

E.3 RUNTIME AND COST ALIGNMENT

Placeholder.

F ADDITIONAL ANALYSES AND NUMERICAL RESULTS

Ablation scope. The ablations compare state-only and joint state-action targets, including a joint + SIGReg control, and vary visual pretraining, history, action context, target parameterization, consistency, and schedule. Each reports both direct and planning-based inference to expose tradeoffs between the two paths.

Raw-result convention. Main-paper figures report aggregate statistics for rapid comparison; this section preserves the observations used to compute them. For local success evaluations, entries are ordered by seed and each percentage is the exact number of successful episodes out of 50. Deterministic measurements, such as FLOPs, are recorded once with their measurement protocol; aggregate probes identify their observational unit and sample count. We do not infer per-run values from means or error bars. For prior-work baselines, we retain only the granularity released by the source and identify the absence of seed-level records explicitly. Any local row lacking its original run vector remains an archival gap until the corresponding job artifact is imported.

F.1 WHY JOINT FLOW MATCHING?

Joint MSE head versus joint flow matching. We replace the conditional action-flow target with an MSE action head while preserving the same encoder, state-action context, joint latent-state/action output, data, and optimization interface. Both variants omit SIGReg; the action-target parameterization is the only controlled difference. The deterministic realization is reported as LeWAM (joint MSE) only to identify the ablation row, not as a separate model.

Table 4: **Output-parameterization ablation.** MSE and flow instantiate the same joint latent-state/action continuation interface. Parenthetical labels identify ablation settings, not separate methods. A dash marks an unaudited checkpoint count.

Configuration	Latent target	Action target	Action selection	Parameters
LeWM (official)	MSE	none	CEM	15M
LeWAM (joint MSE)	MSE	MSE	direct	16M
LeWAM (joint flow)	predictive	conditional flow	direct / planning	–

Table 5: **Verified short-horizon goal-reaching records for MSE target parameterizations.** Values are mean success rate $\pm$ population standard deviation. *Cube* uses the corrected end-effector-aware (+EEF) predicate; its legacy object-only joint-MSE result is omitted. The LeWM row retains its released anti-collapse training and is therefore descriptive, not the matched no-SIGReg control.

Method	Two-Room	Reacher	Push-T	Cube +EEF	Mean
LeWM (latent MSE + CEM)	90.0 $\pm$ 2.0	84.7 $\pm$ 3.1	84.7 $\pm$ 4.2	56.7 $\pm$ 6.1	79.0
LeWAM (joint MSE)	100.0$\pm$0.0	98.7$\pm$1.9	88.0$\pm$4.3	–	–

Table 6: **Why does joint flow work without auxiliary anti-collapse?** The final joint-flow LeWAM is defined without SIGReg; its joint-MSE ablation follows the same no-SIGReg setting. The action-target parameterization is therefore the controlled difference. Dashes reserve measurements that have not yet been imported from verified run artifacts.

Configuration	Params	z_{std}	r_{eff}	Probe R^2	Pixel MSE	Direct SR	Planning SR
LeWAM (joint MSE; no SIGReg)	–	–	–	–	–	–	–
LeWAM (joint flow; no SIGReg)	–	–	–	–	–	–	–

Question 1: why does joint flow work without SIGReg when MSE does not? The final LeWAM realization contains no SIGReg; its joint-MSE ablation is trained under the same no-SIGReg objective. The action target is the controlled difference: conditional-mean MSE versus conditional flow

matching. We hold the encoder, joint state-action interface, data, optimization budget, and evaluation protocol fixed, while reporting head-specific parameter counts. The diagnostic is functional rather than marginal: latent spread and covariance effective rank are paired with observed-state probes, a stop-gradient pixel decoder, direct control, and imagined planning. Planning success is the strongest test of whether the representation remains usable as a world model.

Action information and collapse risk. Let O denote visual history, C the remaining history and task context, $Z = \text{Enc}(O)$, and A the future action or action chunk. For an expressive conditional action model under log loss, the Bayes risks of a constant encoder and an action-sufficient encoder are

$$\mathcal{R}_{\text{coll}} = H(A | C), \quad \mathcal{R}_{\text{suff}} = H(A | O, C), \quad \mathcal{R}_{\text{coll}} - \mathcal{R}_{\text{suff}} = I(A; O | C). \quad (13)$$

Whenever $I(A; O | C) > 0$, visual observations contain action information unavailable from the nonvisual context, so a constant visual encoder cannot minimize the action component of the joint likelihood. This is the conditional *action-identifiability* assumption illustrated in Figure 5.

Conditional-flow risk. For the action coordinate, consider the rectified path $A_s = (1 - s)\epsilon + sA$ with $\epsilon \sim \mathcal{N}(0, I)$ and target velocity $U = A - \epsilon$. The population-optimal vector field and its minimum squared risk are

$$v_s^*(A_s, Z, C) = \mathbb{E}[U | A_s, Z, C], \quad \mathcal{R}_{\text{FM}}(Z) = \mathbb{E}[\text{tr Var}(U | A_s, Z, C)]. \quad (14)$$

By the law of total variance, replacing Z by a constant z_* incurs the nonnegative gap

$$\begin{aligned} \Delta_{\text{FM}} &:= \mathcal{R}_{\text{FM}}(z_*) - \mathcal{R}_{\text{FM}}(Z) \\ &= \mathbb{E}[\|\mathbb{E}[U | A_s, Z, C] - \mathbb{E}[U | A_s, C]\|_2^2] \geq 0. \end{aligned} \quad (15)$$

The inequality is strict when the visual latent disambiguates conditional action flows on a set of nonzero measure. In this regime, action-relevant representation separation is an emergent property of the joint flow likelihood rather than an independently designed constraint; no global latent marginal is prescribed.

Scope of the argument. The analysis does not require every observation to receive a distinct code. States with the same $p(A | O, C)$ may share a representation, while the latent-prediction coordinate preserves information needed for future dynamics. It also does not exclude partial collapse, arbitrary rescaling, or shortcuts through a goal, proprioception, or action history. We therefore test representation sufficiency directly rather than treating marginal variance as a complete collapse criterion.

Why the target parameterization matters. Let o denote visual history, c the remaining context, $z = \text{Enc}(o)$, and a a future action chunk. Under squared error, the population-optimal action head is the conditional mean,

$$m^*(z, c) = \mathbb{E}[a | z, c], \quad \Delta_{\text{MSE}} = \mathbb{E}[\|\mathbb{E}[a | z, c] - \mathbb{E}[a | c]\|_2^2]. \quad (16)$$

Consequently, Δ_{MSE} can vanish even when $p(a | o, c) \neq p(a | c)$, provided that the conditional means coincide. Conditional flow matching instead estimates the velocity field along noisy action interpolants. Its risk gap in Equation 15 can remain positive when observations distinguish modes or other distributional structure beyond the mean. This supplies a stronger action-dependent representation signal for multimodal chunks, but not an unconditional guarantee against collapse: the signal disappears when visual history adds no action information or when another condition bypasses the visual latent.

F.2 WHY END-TO-END JOINT PREDICTION?

We enable SIGReg in both systems and compare latent-only LeWM with joint LeWAM using MSE targets throughout. Predictor width is adjusted to match total trainable parameters; data, encoder, optimizer, updates, and evaluation budgets are shared. This control removes target-density and anti-collapse differences, leaving the joint action target as the structural change. The verified records in Table 5 remain descriptive until the exact parameter-matched runs in Table 7 are frozen.

Table 7: **What is gained by end-to-end joint prediction?** Both rows use MSE targets and SIGReg, with total trainable parameters, data, encoder, optimizer, updates, and evaluation budget matched. The action target is the controlled difference. Dashes denote pending measurements rather than estimated values.

Configuration	Params	z_{std}	r_{eff}	Probe R^2	Pixel MSE	Planning SR	Direct SR
LeWM (latent MSE + SIGReg)	–	–	–	–	–	–	n/a
LeWAM (joint MSE + SIGReg)	–	–	–	–	–	–	–

F.3 BEHAVIOR CLONING

We evaluate the same LeWAM backbone as a pure behavior-cloning policy. The model is trained to predict demonstrated action chunks from the benchmark conditioning inputs and is executed without latent prediction, imagination, or test-time planning. *Placeholder*.

F.4 CONTACT-SENSITIVE LATENT PLANNING

Purpose. Section 3 gives the operational motivation for joint state–action prediction in contact-rich control. This appendix supplies the technical basis for that motivation: contact can abruptly change rollout sensitivities even when observations and latent states remain visually continuous. It does not introduce a separate planner.

Hybrid sensitivity. Contact-rich manipulation (Posa et al., 2014; Suh et al., 2022) can be described by interaction modes m , such as free motion, sticking, and sliding. Within one mode the state follows a smooth vector field; a contact event activates a new mode and may apply a reset map:

$$\dot{\mathbf{x}} = f_m(\mathbf{x}, \mathbf{a}), \quad g_{m \rightarrow n}(\mathbf{x}) = 0, \quad \mathbf{x}^+ = R_{m \rightarrow n}(\mathbf{x}^-). \quad (17)$$

Configuration can remain visually continuous while impact changes velocity or a new contact constraint changes how actions affect subsequent motion. Perturbations across the event are therefore propagated by a saltation matrix (Burden et al., 2016; Kong et al., 2024) rather than the within-mode Jacobian:

$$\delta \mathbf{x}^+ = \Xi_{m \rightarrow n} \delta \mathbf{x}^-, \quad \delta \mathbf{z}^+ = D(\text{Enc} \circ \text{Obs})(\mathbf{x}^+) \Xi_{m \rightarrow n} \delta \mathbf{x}^-. \quad (18)$$

Thus contact does not require a discontinuous representation. Instead, neighboring action sequences can cross an event at different times or enter different modes, producing sharply different post-contact rollouts.

Implication for latent goal costs. Euclidean latent planning requires candidate outcomes to be ordered by task progress. A sufficient ordering condition is

$$c_{\text{task}}(\mathbf{o}_i, \mathbf{o}_g) < c_{\text{task}}(\mathbf{o}_j, \mathbf{o}_g) \implies d_{\text{lat}}(\mathbf{z}_i, \mathbf{z}_g) < d_{\text{lat}}(\mathbf{z}_j, \mathbf{z}_g). \quad (19)$$

Latent planners such as DINO-WM (Zhou et al., 2024), LeWM (Maes et al., 2026b), PLDM (Sobal et al., 2025), and V-JEPA 2-AC (Assran et al., 2025) instantiate this surrogate with embedding distance. Because contact can change the reachable set much more sharply than visual appearance, Euclidean representation distance need not match dynamical reachability (Hartikainen et al., 2020; Tian et al., 2021; Wang et al., 2023).

Diagnostic. We test whether latent goal cost preserves the ordering of controllable task progress before, during, and after contact, using rank correlation and pairwise ordering accuracy against simulator progress. *Placeholder*.

F.5 ROBUSTNESS AND REPRESENTATION

Temporal and Controllable Structure. We test whether action prediction changes the geometry of the learned representation using normalized trajectory curvature, Euclidean-to-geodesic distortion, local action Jacobians, and action-shuffle sensitivity. The same probes are evaluated for latent-only, split, and joint predictors under matched encoders and capacity. *Placeholder*.

Controlled Configuration Shifts. We vary layout, appearance, object configuration, and action-relevant geometry while keeping task semantics fixed. Results report absolute success, degradation

from the in-distribution reference, and mechanism probes for every shift family. This design separates general robustness from gains confined to one axis. *Placeholder*.

Task and Data Scale. We scale the joint objective across task families, trajectory horizons, and mixtures of demonstrations with suboptimal behavior. Dynamics prediction is evaluated separately from the action marginal because suboptimal transitions remain valid dynamics evidence while providing weaker goal-directed action supervision. *Placeholder*.

Representation mechanism probes. Final matched probe measurements and their observational units will be reported here. *Placeholder*.

F.6 QUALITATIVE RESULTS AND FAILURE CASES

The final supplement will pair state-action rollouts and representative failures with the numerical records above. Robot evaluations will state the number of trials and task-level success alongside every qualitative sequence.

G REAL-ROBOT SYSTEM: ARCHITECTURE, PLATFORM, AND DATA

This section records the robot-specific architecture, platform, data collection, and evaluation details supplied on August 6, 2026. Training and inference otherwise follow the shared LeWAM methodology in Section 3; only deviations required by the physical observation and control interface are documented here.

G.1 FOUNDATION-BACKED JOINT PREDICTOR

Scope. The real-robot study tests whether a strong video foundation model can transfer visual and temporal priors into a representation-space world action model. A shared, frozen V-JEPA 2.1 (Murlabadia et al., 2026) backbone supplies the video representation; trainable attentive pooling, stream projections, and a Transformer predictor adapt that representation to robot-specific state-action continuations. The implementation uses the same configurable network for three model families: predicting only actions gives a policy, predicting only future camera representations gives an action-conditioned world model, and predicting both gives the real-robot LeWAM configuration.

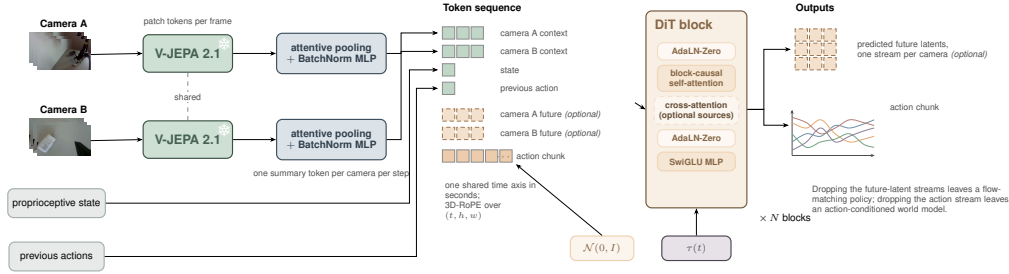


Figure 12: Real-robot multi-stream architecture. Two camera streams share a frozen video backbone; trainable pooling maps each camera and timestep to a summary token. Proprioception, previous actions, future camera representations, and an action chunk occupy a common token sequence processed by one DiT trunk.

Visual and proprioceptive inputs. Each camera frame is encoded by the frozen video backbone and reduced to one summary token per camera and timestep by attentive pooling followed by a BatchNorm MLP. Proprioceptive state and previous actions enter as raw vectors and are linearly projected to the model dimension. Future camera summaries and action tokens occupy separate predicted-stream positions initialized from Gaussian noise. The stream declarations, rather than separate model implementations, determine which inputs are consumed and which outputs are generated.

Temporal specification. Streams are placed on one physical time axis in seconds, which permits camera and control signals to use different rates without hard-coded index arithmetic. Table 8 transcribes the concrete configuration included in the supplied implementation report; Figure 13 visualizes its temporal geometry.

Table 8: Stream configuration in the August 6 implementation snapshot. Indices are defined at each stream’s own sampling rate.

Stream	Role	Dimension	Temporal specification	Rate
Scene-camera context	condition	512	encoded $-1:0$; raw $-18, -12, -6, 0$	2.5 Hz
Proprioceptive position	condition	7	0	30 Hz
Future camera summary	prediction	512	1:3	2.5 Hz
Action chunk	prediction	7	0:35	30 Hz

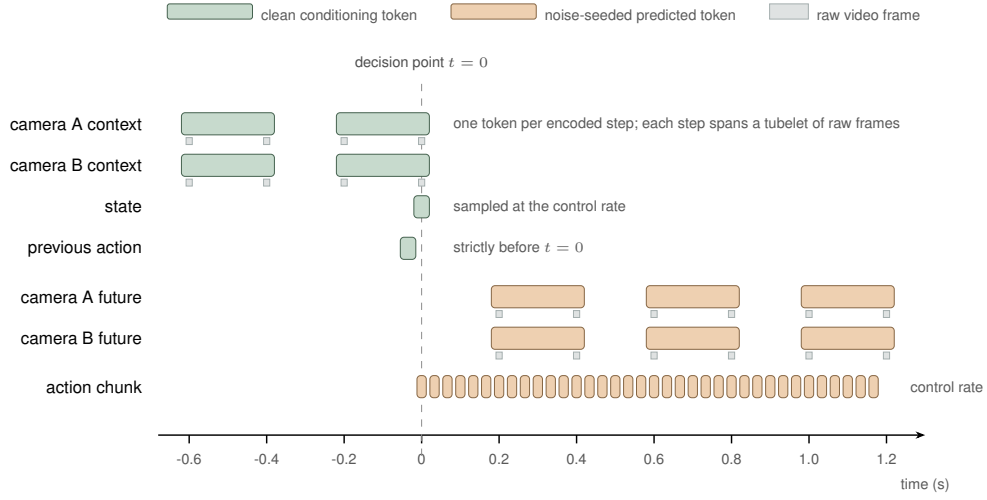


Figure 13: Conditioning and prediction streams on the shared time axis. Green blocks denote clean conditioning tokens, orange blocks denote noise-seeded prediction tokens, and narrow gray blocks denote raw video frames.

Attention and position. Self-attention is block causal: a key is visible when its causal block begins before the query block ends. Conditioning tokens act only as clean key-value context because no prediction head reads their outputs, so they never attend to noise-seeded prediction tokens even when time blocks overlap. Positions are encoded in time, row, and column with rotary embeddings (Ma et al., 2024); each Transformer sublayer is modulated by zero-initialized adaptive LayerNorm (Peebles & Xie, 2023) at the flow timestep. Figure 14 shows the resulting mask for the illustrative stream specification.

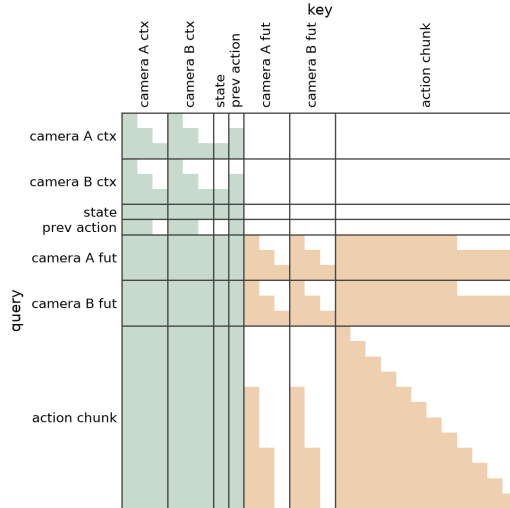


Figure 14: Block-causal attention mask generated from the stream specification. Colored cells mark permitted attention, with green keys for clean conditioning streams and orange keys for predicted streams.

Table 9: Configurations instantiated by the shared real-robot architecture. Exact trainable parameter counts remain to be frozen; current runs lie in the approximate 100–200M-parameter range.

Configuration	Predicted streams	Tokens	Action chunk
Policy	action	43	36
World action model	action + camera futures	49	36
World model	camera futures	13	–

G.2 IMPLEMENTATION DETAILS

The real-robot model follows the joint flow-matching training and frozen-predictor inference methodology in Section 3, without a separate robot-specific objective or post-trained action model. Its only structural changes are the physical input and output interfaces described above: scene and wrist camera views, proprioceptive state, and robot action chunks replace the corresponding simulation streams. *Implementation placeholder.*

G.3 ROBOT PLATFORM AND DATA COLLECTION

Teleoperation hardware. Demonstrations are collected by joint-space teleoperation from a reBot Arm 102 leader to a reBot B601-DM follower through LeRobot (Cadene et al., 2026). The implementation adds a follower driver, gravity model, and stereo-camera backend. The follower runs in impedance mode with gravity feedforward from the robot model. This supports the arm’s weight and reduces persistent offset between commanded and measured joint positions.



Figure 15: Leader-follower teleoperation pair at the calibration zero pose. Images are courtesy of Seed Studio.

Cameras and recording. A ZED-Mini observes the workspace at 672×376 pixels per eye, with both eyes recorded from one capture. A wrist-mounted Logitech C270 records 640×360 video. The three video columns and the control loop run at 30 Hz. A headless Raspberry Pi 5 recording process stores the seven-dimensional leader-derived joint target as the action and follower position, torque, and velocity as a 21-dimensional state. Between episodes, the follower ramps to its calibration zero pose and waits for the leader to be returned to the same pose, preventing an uncontrolled jump at the beginning of a recording.

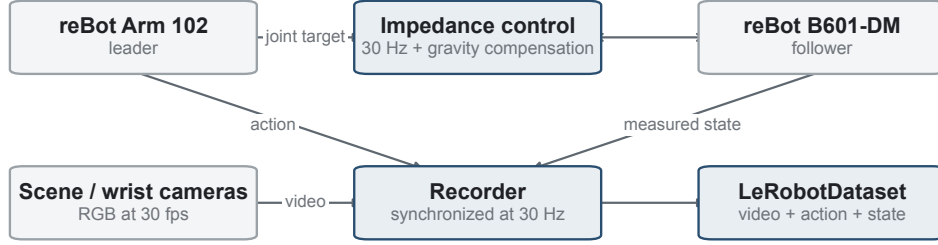


Figure 16: Teleoperation and synchronized data collection. The leader supplies joint targets; the controller exchanges commands and feedback with the gravity-compensated follower; scene video, wrist video, actions, and measured state are recorded at 30 Hz.

Table 10: Single-task datasets in the supplied report. Collected datasets use 7D actions, 21D recorded state, three camera columns, and a 30 Hz recording rate.

Task	Episodes	Frames	Duration	Mean episode	Cameras
Charging block into bin	200	61,466	34 min	10.2 s	3
Towel fold	100	275,323	153 min	91.8 s	3
Color sorting into bins			<i>planned</i>		
Tape roll onto charging block			<i>planned</i>		
Real-world Push-T			<i>planned</i>		

Task coverage. The collected tasks span rigid pick-and-place and deformable manipulation. Towel-fold episodes are approximately nine times longer on average than the charging-block episodes and introduce extended contact with deformable material. The planned physical Push-T task uses the 3D-printed object shown in Figure 17.



Figure 17: 3D-printed object prepared for the planned physical Push-T task.

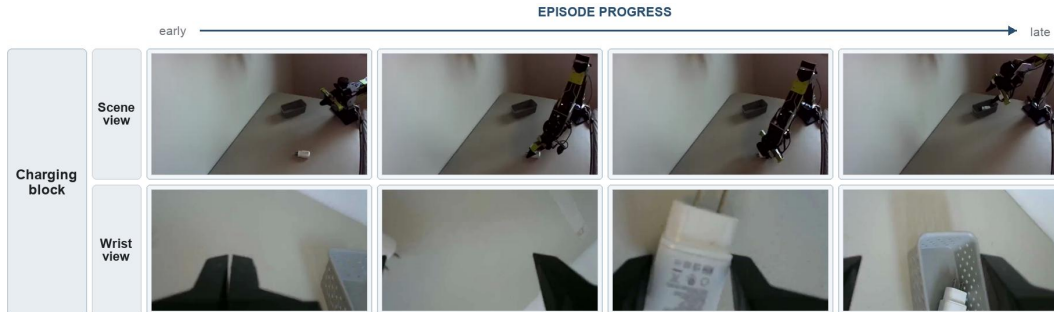


Figure 18: Synchronized scene and wrist views from a charging-block demonstration. Frames advance from early to late within the same episode.

G.4 REPRESENTATION DIAGNOSTICS AND EVALUATION

Pixel-decoder probe. A separate probe tests what visual information remains in a frozen representation. Learnable per-patch queries read a frozen encoder output through cross-attention layers and project it to pixel patches; only this decoder is optimized. The architecture of the diagnostic is shown in Figure 19.

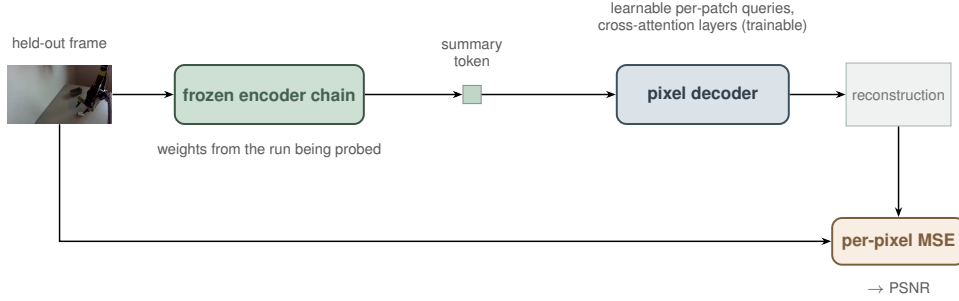


Figure 19: Frozen-encoder pixel-decoder probe. The target frame and reconstruction are compared with per-pixel MSE, from which PSNR is computed.

Preliminary decoding snapshot. The supplied report records a 14×14 patch grid from a frozen V-JEPA 2.1 ViT-B decoded to 224×224 pixels on held-out towel-fold episodes. After 5,000 decoder steps, the report gives a held-out PSNR of 28.2 dB. This value is retained as a preliminary implementation diagnostic rather than as evidence of closed-loop task success.



Figure 20: Preliminary held-out towel-fold decoding. The upper row contains target frames; the lower row contains reconstructions from the frozen patch grid.

Closed-loop evaluation. Rollouts execute on the same physical rig at 30 Hz. The robot sends its observation window to a remote inference machine, receives an action chunk, and executes it under the receding-horizon schedule. Episode outcomes are labeled success or failure by a human evaluator. Final reporting will retain every trial outcome, include binomial confidence intervals, and separate model-only latency from network-inclusive control-cycle latency.

Causal study. The planned primary ablation is a 2×2 comparison of backbone source and predictive target. Replacing pretrained V-JEPA 2.1 with an identically frozen random initialization removes transferred video knowledge while preserving downstream capacity and optimization. Removing future-camera prediction from the same Transformer produces an action-only flow model with the same action representation and execution schedule. The main effects and interaction isolate video-prior transfer, joint state-action prediction, and whether the two reinforce one another.

Reporting contract. Every final configuration will report immutable train and evaluation splits, task-level trial counts, action-chunk horizon, flow-integration steps, exact frozen and trainable pa-

parameter counts, model-only latency, and end-to-end control latency. Scaling curves will use frames or recorded hours because episode duration differs substantially across tasks. Visual, object, and initial-state shifts will be specified separately rather than merged into one OOD label.

H RECOMMENDATIONS FOR FUTURE WORK

Scaling test-time planning with world action models. CEM (Rubinstein, 1999) and MPPI (Williams et al., 2017) repeatedly sample, roll out, and rank action sequences at every decision. Gradient planning (Jyothir et al., 2023; Parthasarathy et al., 2025) reduces the number of candidates, but exposes the optimizer directly to the action coordinates and can require planning-aware model training. Joint imagination instead uses the frozen action conditional to parameterize candidate behavior, and Steer-MPC searches over bounded goal-latent offsets while regenerating actions through that conditional. A natural extension is to optimize or sample several such offsets in parallel, then select among the induced state-action continuations using inference-only information such as safety constraints or language instructions.

Integrating reinforcement learning. LeWAM trains on reward-free trajectories, which leaves reward and value outside its objective. Adding them has a natural point of entry, because the predictor already imagines states and actions together, so a value or reward signal can shape the same continuation that will be executed, and policy improvement can happen inside imagination through the interface the model already exposes. A future extension could attach reward or value prediction to the joint continuation and evaluate whether policy improvement can share this interface with model-based agents such as Dreamer (Hafner et al., 2020) and TD-MPC2 (Hansen et al., 2024), rather than introducing an unrelated policy distillation objective.

Unified action embeddings. The action-space extension discussed in Section F.5 is the second direction we consider open: mapping heterogeneous task and embodiment action spaces into one shared representation would avoid regression across incompatible controls, at the cost of a tokenizer and an execution decoder that this paper does not build.